

```
----- 第 1 页 -----
// ===== FILE: Assets/Scripts/Core/SaveData.cs =====
using System;
using System.Collections.Generic;

/// <summary>JsonUtility 可序列化的 string→int 条目。</summary>
[Serializable]
public class StringIntEntry
{
    public string id;
    public int value;
}

/// <summary>JsonUtility 可序列化的 string id 条目。</summary>
[Serializable]
public class StringIdEntry
{
    public string id;
}

/// <summary>JsonUtility 可序列化的 int id 条目。</summary>
[Serializable]
public class IntIdEntry
{
    public int id;
}

[Serializable]
public class SaveData
{
    // == 货币 / 资源（统一上限见 ResourceWallet，体力有特殊上限） ==
    public long totalGold = 0;
    public int talentPoints = 0;
    public int diamond = 0;
    public int enchantStones = 0;    // 附魔石
    public int decomposeMats = 0;    // 分解材料
    /// <summary>体力（特殊上限 GameConfig.STAMINA_MAX）</summary>
    public int stamina = 100;
    /// <summary>体力上次结算 Unix 秒（用于回复）</summary>
    public long lastStaminaUtc = 0;
    /// <summary>广告奖励日键 yyyyMMdd（UTC）</summary>
    public string adRewardDayKey = "";
    /// <summary>当日已领体力广告次数</summary>
    public int adStaminaClaimCount = 0;
    /// <summary>当日已领金币广告次数</summary>
    public int adGoldClaimCount = 0;
    /// <summary>邮件箱（资源溢出等）</summary>
    public List<MailEntry> mailInbox = new List<MailEntry>();

    // == 天赋（List 持久化；运行时用 Dictionary） ==
    public List<StringIntEntry> talentEntries = new List<StringIntEntry>();
}
```

----- 第 2 页 -----

```
[NonSerialized] public Dictionary<string, int> talents = new Dictionary<string, int>();

// === 传说武器 ===
public List<StringIdEntry> unlockedLegendaryWeaponEntries = new List<StringIdEntry>();
[NonSerialized] public HashSet<string> unlockedLegendaryWeapons = new HashSet<string>();

// === 遗产装备 ===
public List<EquipmentData> legacyEquipPool = new List<EquipmentData>();

// === 城镇等级 ===
public TownLevel townLevel = new TownLevel();

/// <summary>公会等级（影响高级佣兵解锁等）</summary>
public int guildLevel = 1;

// === 佣兵 ===
public List<MercenaryData> permanentMerces = new List<MercenaryData>();

// === 玩家基础属性 ===
public int playerStrength = 0;    // 额外力量（天赋/遗产加成）
public int playerIntelligence = 0; // 额外智力
public int playerAgility = 0;    // 额外敏捷
public int playerVitality = 0;   // 额外体质

// === 成就系统 ===
public List<StringIntEntry> achievementProgressEntries = new List<StringIntEntry>();
[NonSerialized] public Dictionary<string, int> achievementProgress = new Dictionary<string, int>();
public List<StringIdEntry> completedAchievementEntries = new List<StringIdEntry>();
[NonSerialized] public HashSet<string> completedAchievements = new HashSet<string>();
public int totalAchievementPoints = 0;
public List<IntIdEntry> claimedMilestoneEntries = new List<IntIdEntry>();
[NonSerialized] public HashSet<int> claimedMilestoneIds = new HashSet<int>();

/// <summary>战前选择的玩家技能 id（PlayerSkillDefs）</summary>
public string selectedPlayerSkillId = "heal_spring";

public bool openingIntroPlayed;
public bool tutorialIntroDone;
public bool tutorialBattleCleared;
public bool tutorialOutroPending;
public bool tutorialDone;
public bool chapter1IntroDone;
public bool chapter1ChoiceDone;
public List<NpcBondEntry> npcBonds = new List<NpcBondEntry>();
public List<StoryChoiceEntry> storyChoices = new List<StoryChoiceEntry>();

// === 章节进度 ===
public int maxUnlockedChapter = 1; // 最大解锁章节
/// <summary>章节通关次数列表（用于渐进式怪物解锁）</summary>
public List<ChapterClearCountEntry> chapterClearCounts = new List<ChapterClearCountEntry>();
```

----- 第 3 页 -----

// === 时间戳 ===

public long lastSaveTime = 0;

/// <summary>JsonUtility 反序列化后调用: List → 运行时 Dictionary/HashSet。</summary>

public void SyncRuntimeFromLists()

{

talentEntries ??= new List<StringIntEntry>();

unlockedLegendaryWeaponEntries ??= new List<StringIdEntry>();

achievementProgressEntries ??= new List<StringIntEntry>();

completedAchievementEntries ??= new List<StringIdEntry>();

claimedMilestoneEntries ??= new List<IntIdEntry>();

mailInbox ??= new List<MailEntry>();

legacyEquipPool ??= new List<EquipmentData>();

permanentMercs ??= new List<MercenaryData>();

npcBonds ??= new List<NpcBondEntry>();

storyChoices ??= new List<StoryChoiceEntry>();

chapterClearCounts ??= new List<ChapterClearCountEntry>();

townLevel ??= new TownLevel();

talents = new Dictionary<string, int>();

for (int i = 0; i < talentEntries.Count; i++)

{

var e = talentEntries[i];

if (e == null || string.IsNullOrEmpty(e.id)) continue;

talents[e.id] = e.value;

}

unlockedLegendaryWeapons = new HashSet<string>();

for (int i = 0; i < unlockedLegendaryWeaponEntries.Count; i++)

{

var e = unlockedLegendaryWeaponEntries[i];

if (e == null || string.IsNullOrEmpty(e.id)) continue;

unlockedLegendaryWeapons.Add(e.id);

}

achievementProgress = new Dictionary<string, int>();

for (int i = 0; i < achievementProgressEntries.Count; i++)

{

var e = achievementProgressEntries[i];

if (e == null || string.IsNullOrEmpty(e.id)) continue;

achievementProgress[e.id] = e.value;

}

completedAchievements = new HashSet<string>();

for (int i = 0; i < completedAchievementEntries.Count; i++)

{

var e = completedAchievementEntries[i];

if (e == null || string.IsNullOrEmpty(e.id)) continue;

completedAchievements.Add(e.id);

----- 第 4 页 -----

```

    }

    claimedMilestoneIds = new HashSet<int>();
    for (int i = 0; i < claimedMilestoneEntries.Count; i++)
    {
        var e = claimedMilestoneEntries[i];
        if (e == null) continue;
        claimedMilestoneIds.Add(e.id);
    }
}

/// <summary>写入 JSON 前调用: 运行时 Dictionary/HashSet → List。</summary>
public void SyncListsFromRuntime()
{
    talents ??= new Dictionary<string, int>();
    unlockedLegendaryWeapons ??= new HashSet<string>();
    achievementProgress ??= new Dictionary<string, int>();
    completedAchievements ??= new HashSet<string>();
    claimedMilestoneIds ??= new HashSet<int>();

    talentEntries = new List<StringIntEntry>(talents.Count);
    foreach (var kv in talents)
        talentEntries.Add(new StringIntEntry { id = kv.Key, value = kv.Value });

    unlockedLegendaryWeaponEntries = new List<StringIdEntry>(unlockedLegendaryWeapons.Count);
    foreach (string id in unlockedLegendaryWeapons)
        unlockedLegendaryWeaponEntries.Add(new StringIdEntry { id = id });

    achievementProgressEntries = new List<StringIntEntry>(achievementProgress.Count);
    foreach (var kv in achievementProgress)
        achievementProgressEntries.Add(new StringIntEntry { id = kv.Key, value = kv.Value });

    completedAchievementEntries = new List<StringIdEntry>(completedAchievements.Count);
    foreach (string id in completedAchievements)
        completedAchievementEntries.Add(new StringIdEntry { id = id });

    claimedMilestoneEntries = new List<IntIdEntry>(claimedMilestoneIds.Count);
    foreach (int id in claimedMilestoneIds)
        claimedMilestoneEntries.Add(new IntIdEntry { id = id });
}

[Serializable]
public class ChapterClearCountEntry
{
    public int chapter;
    public int clearCount;
}

[Serializable]
```

----- 第 5 页 -----

```
public class TownLevel
```

```
{
    public int blacksmith = 1;
    public int tavern = 1;
    public int altar = 1;
    public int farm = 1;
}
```

```
[Serializable]
```

```
public class EquipmentData
```

```
{
    public string equipId;
    public int rarity;
    public int star;
    public int requireLevel;
    public List<AttrBonusData> attrBonus = new List<AttrBonusData>();
    public List<string> tags = new List<string>();
    public bool isLegacy;
}
```

```
[Serializable]
```

```
public class AttrBonusData
```

```
{
    public AttrType attrType;
    public float value;
    public bool isPercent;
}
```

```
[Serializable]
```

```
public class MercenaryData
```

```
{
    /// <summary>形象/预制体模板 ID（如 gongshou101），可重复招募同一模板</summary>
    public string mercId;
    /// <summary>展示姓名（同形象可不同名）</summary>
    public string displayName;
    /// <summary>实例唯一 ID，名册可有多条同 mercId</summary>
    public string uid;
    public int favorLevel;
    public int level;
    /// <summary>星级 1~5</summary>
    public int star = 1;
    /// <summary>佩戴技能（Ally 技能 id，如 ally_heal）</summary>
    public string skillId;
}
```

```
// ===== FILE: Assets/Scripts/Systems/MercenaryOfferGenerator.cs =====
```

```
using System.Collections.Generic;
```

```
using UnityEngine;
```

```
/// <summary>
```

----- 第 6 页 -----

/// 酒馆招募：从佣兵形象池抽 3 个候选项。

/// 形象（prefab/mercId）可相同，但姓名、等级、星级、技能必须做出差异。

/// </summary>

```
public static class MercenaryOfferGenerator
{
    public const int OfferCount = 3;

    static readonly string[] NamePool =
    {
        "阿雷", "薇拉", "石盾", "迅羽", "灰烬", "露娜", "铁腕", "影刺",
        "凯恩", "茉拉", "雷恩", "希雅", "布鲁", "诺拉", "达克", "艾琳",
        "霍克", "莉娜", "加仑", "苏尔", "米娅", "托尔", "茜茜", "沃克"
    };

    static readonly string[] AllySkillPool =
    {
        "ally_heal", "ally_shield", "ally_atk_up", "ally_atk_speed", "ally_crit_up", "ally_thunder"
    };

    static readonly string[] SkillDisplayNames =
    {
        "治愈之泉", "圣盾壁垒", "战意爆发", "疾风架势", "致命专注", "天雷裁决"
    };

    public static string SkillDisplayName(string skillId)
    {
        if (string.IsNullOrEmpty(skillId)) return "未知技能";
        for (int i = 0; i < AllySkillPool.Length; i++)
        {
            if (AllySkillPool[i] == skillId)
                return SkillDisplayNames[i];
        }
        var cfg = SkillRegistry.Instance != null ? SkillRegistry.Instance.Get(skillId) : null;
        if (cfg != null && !string.IsNullOrEmpty(cfg.skillName)) return cfg.skillName;
        return skillId;
    }

    /// <summary>生成三选一；池为空时回退初级模板列表。</summary>
    public static List<MercenaryData> GenerateOffers()
    {
        var pool = GetVisualPool();
        var offers = new List<MercenaryData>(OfferCount);
        var usedSignatures = new HashSet<string>();

        for (int i = 0; i < OfferCount; i++)
        {
            MercenaryData offer = null;
            for (int attempt = 0; attempt < 24; attempt++)
            {
                offer = RollOne(pool);
            }
        }
    }
}
```

----- 第 7 页 -----

```

        string sig = Signature(offer);
        if (!usedSignatures.Contains(sig))
        {
            usedSignatures.Add(sig);
            break;
        }
    }
    if (offer == null) offer = RollOne(pool);
    // 强制本批姓名互异
    EnsureUniqueName(offer, offers);
    offers.Add(offer);
}
return offers;
}

static void EnsureUniqueName(MercenaryData offer, List<MercenaryData> existing)
{
    if (offer == null) return;
    bool clash;
    int guard = 0;
    do
    {
        clash = false;
        for (int i = 0; i < existing.Count; i++)
        {
            if (existing[i] != null && existing[i].displayName == offer.displayName)
            {
                clash = true;
                offer.displayName = NamePool[Random.Range(0, NamePool.Length)] + Random.Range(1, 99);
                break;
            }
        }
        guard++;
    } while (clash && guard < 20);
}

static string Signature(MercenaryData m)
{
    if (m == null) return "";
    return $"{m.mercId} | {m.displayName} | {m.level} | {m.star} | {m.skillId}";
}

static List<string> GetVisualPool()
{
    var result = new List<string>();
    var mm = MercenaryManager.Instance;
    if (mm != null)
    {
        var hireable = mm.GetHireableMercIds();
        if (hireable != null)
    }

```

----- 第 8 页 -----

```

    {
        for (int i = 0; i < hireable.Count; i++)
        {
            if (!string.IsNullOrEmpty(hireable[i]))
                result.Add(hireable[i]);
        }
    }
}
if (result.Count == 0)
{
    // 回退：初级形象模板（非全弓手）
    result.AddRange(new[]
    {
        "dunbing101", "gongshou101", "kuangzhan101", "naima101", "qital01",
        "dunbing102", "kuangzhan102", "naima102"
    });
}
return result;
}

static MercenaryData RollOne(List<string> pool)
{
    string mercId = pool[Random.Range(0, pool.Count)];
    int level = Random.Range(1, 11); // 1~10
    int star = Random.Range(1, 6); // 1~5
    string skillId = AllySkillPool[Random.Range(0, AllySkillPool.Length)];
    string name = NamePool[Random.Range(0, NamePool.Length)];
    // 同批再抽时略微拉开：星级高的略抬等级
    if (star >= 4) level = Mathf.Max(level, Random.Range(5, 11));

    return new MercenaryData
    {
        mercId = mercId,
        displayName = name,
        uid = System.Guid.NewGuid().ToString("N"),
        favorLevel = 1,
        level = level,
        star = star,
        skillId = skillId
    };
}

public static string FormatCard(MercenaryData m)
{
    if (m == null) return "空";
    string job = MercenaryManager.Instance != null
        ? MercenaryManager.Instance.GetJobName(m.mercId)
        : m.mercId;
    string stars = new string('★', Mathf.Clamp(m.star, 1, 5));
    return $"{m.displayName}\n{job}\nLv {Mathf.Max(1, m.level)} {stars}\n技能: {SkillDisplayName(m.skillId)}";
}

```


----- 第 9 页 -----

```

    }

    public static string FormatRosterLine(MercenaryData m, bool deploy)
    {
        if (m == null) return "";
        string name = string.IsNullOrEmpty(m.displayName) ? m.mercId : m.displayName;
        string job = MercenaryManager.Instance != null
            ? MercenaryManager.Instance.GetJobName(m.mercId)
            : m.mercId;
        string stars = new string('★', Mathf.Clamp(m.star < 1 ? 1 : m.star, 1, 5));
        return $" {(deploy ? "★出战" : "•待命")} {name} ({job}) Lv{Mathf.Max(1, m.level)} {stars}
[{SkillDisplayName(m.skillId)}]";
    }
}

// ===== FILE: Assets/Scripts/UI/TavernRosterPanel.cs =====
using System.Collections.Generic;
using System.Text;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// 酒馆佣兵招募：从形象池三选一。
/// 同形象可反复招募，但姓名/等级/星级/技能不同。
/// UI 代码搭建；酒馆场景预制体资源由用户自行替换，本面板不覆盖预制体。
/// </summary>
public class TavernRosterPanel : MonoBehaviour
{
    static TavernRosterPanel _instance;
    GameObject _root;
    Text _body;
    Text _title;
    readonly List<Image> _cardBgs = new List<Image>();
    readonly List<Text> _cardLabels = new List<Text>();
    readonly List<Image> _cardIcons = new List<Image>();
    List<MercenaryData> _offers = new List<MercenaryData>();
    int _selected;

    public static void Show()
    {
        Ensure().Open();
    }

    static TavernRosterPanel Ensure()
    {
        if (_instance != null) return _instance;
        var go = new GameObject("TavernRosterPanel");
        DontDestroyOnLoad(go);
        _instance = go.AddComponent<TavernRosterPanel>();
        return _instance;
    }
}

```

----- 第 10 页 -----

```
void Awake() => _instance = this;

void OnDestroy()
{
    if (_instance == this) _instance = null;
}

void Open()
{
    TownSaveAlign.AlignAll();
    BuildIfNeeded();
    RerollOffers();
    Refresh();
    if (_root != null)
    {
        _root.SetActive(true);
        _root.transform.SetAsLastSibling();
    }
    GameFonts.ApplyToHierarchy(transform);
}

void RerollOffers()
{
    _offers = MercenaryOfferGenerator.GenerateOffers();
    _selected = 0;
}

void Refresh()
{
    var data = SaveSystem.Instance?.Data;
    var mm = MercenaryManager.Instance;
    int slots = mm != null
        ? mm.GetMaxMercSlots()
        : Mathf.Clamp(data?.townLevel?.tavern ?? 1, 0, 2);

    var sb = new StringBuilder();
    sb.AppendLine($"【佣兵名册】出战槽 {slots} (前 {slots} 名出战)");
    sb.AppendLine();

    var list = data?.permanentMercs;
    if (list == null || list.Count == 0)
        sb.AppendLine("暂无永久佣兵。下方三选一招募。");
    else
    {
        for (int i = 0; i < list.Count; i++)
            sb.AppendLine(MercenaryOfferGenerator.FormatRosterLine(list[i], i < slots));
    }

    var active = mm != null ? mm.GetActiveMercIds() : new List<string>();
```

----- 第 11 页 -----

```
sb.AppendLine();
sb.AppendLine($"开战将带出模板: {(active.Count > 0 ? string.Join("、", active) : "(无)")}");
if (_body != null) _body.text = sb.ToString();

RefreshCards();
}

void RefreshCards()
{
    for (int i = 0; i < _cardLabels.Count; i++)
    {
        MercenaryData offer = (i < _offers.Count) ? _offers[i] : null;
        if (_cardLabels[i] != null)
            _cardLabels[i].text = offer != null ? MercenaryOfferGenerator.FormatCard(offer) : "—";
        if (_cardBgs[i] != null)
            _cardBgs[i].color = (i == _selected)
                ? new Color(0.55f, 0.42f, 0.22f, 1f)
                : new Color(0.28f, 0.22f, 0.18f, 1f);
        if (_cardIcons[i] != null && offer != null && MercenaryManager.Instance != null)
        {
            var icon = MercenaryManager.Instance.GetIcon(offer.mercId);
            _cardIcons[i].sprite = icon;
            _cardIcons[i].enabled = icon != null;
            _cardIcons[i].preserveAspect = true;
        }
    }
}

void Select(int idx)
{
    if (idx < 0 || idx >= _offers.Count) return;
    _selected = idx;
    RefreshCards();
}

void OnConfirmRecruit()
{
    var data = SaveSystem.Instance?.Data;
    if (data == null)
    {
        UIManager.Instance?.ShowToast("存档未就绪");
        return;
    }
    if (_offers == null || _selected < 0 || _selected >= _offers.Count || _offers[_selected] == null)
    {
        UIManager.Instance?.ShowToast("请先选择一名佣兵");
        return;
    }

    if (data.permanentMercs == null)
```

----- 第 12 页 -----

```

        data.permanentMerces = new List<MercenaryData>();

        var picked = CloneOffer(_offers[_selected]);
        if (string.IsNullOrEmpty(picked.uid))
            picked.uid = System.Guid.NewGuid().ToString("N");
        if (picked.star < 1) picked.star = 1;
        if (picked.level < 1) picked.level = 1;
        if (string.IsNullOrEmpty(picked.skillId))
            picked.skillId = SkillRegistry.DefaultMercMeleeSkillId;

        data.permanentMerces.Add(picked);
        if (data.townLevel == null) data.townLevel = new TownLevel();
        if (data.townLevel.tavern < 1) data.townLevel.tavern = 1;
        SaveSystem.Instance.Save();

        string name = string.IsNullOrEmpty(picked.displayName) ? picked.mercId : picked.displayName;
        UIManager.Instance?.ShowToast($"已招募: {name}");
        RerollOffers();
        Refresh();
    }

    static MercenaryData CloneOffer(MercenaryData src)
    {
        return new MercenaryData
        {
            mercId = src.mercId,
            displayName = src.displayName,
            uid = src.uid,
            favorLevel = src.favorLevel,
            level = src.level,
            star = src.star,
            skillId = src.skillId
        };
    }

    void BuildIfNeeded()
    {
        if (_root != null) return;
        var canvas = gameObject.GetComponent<Canvas>() ?? gameObject.AddComponent<Canvas>();
        if (gameObject.GetComponent<GraphicRaycaster>() == null)
            gameObject.AddComponent<GraphicRaycaster>();
        UICanvasSetup.Apply(canvas);
        canvas.sortingOrder = 900;

        _root = new GameObject("Root", typeof(RectTransform));
        _root.transform.SetParent(transform, false);
        Stretch(_root.GetComponent<RectTransform>());

        var dim = new GameObject("Dim", typeof(RectTransform), typeof(Image), typeof(Button));
        dim.transform.SetParent(_root.transform, false);
    }

```

----- 第 13 页 -----

```

Stretch(dim.GetComponent<RectTransform>());
dim.GetComponent<Image>().color = new Color(0f, 0f, 0f, 0.55f);
dim.GetComponent<Button>().onClick.AddListener(Close);

var panel = new GameObject("Panel", typeof(RectTransform), typeof(Image));
panel.transform.SetParent(_root.transform, false);
var prt = panel.GetComponent<RectTransform>();
prt.anchorMin = prt.anchorMax = new Vector2(0.5f, 0.5f);
prt.sizeDelta = new Vector2(640f, 620f);
panel.GetComponent<Image>().color = new Color(0.14f, 0.1f, 0.08f, 0.97f);

_title = MakeText(panel.transform, "酒馆 • 佣兵招募 (三选一)", 26);
var trt = _title.rectTransform;
trt.anchorMin = new Vector2(0.05f, 0.91f);
trt.anchorMax = new Vector2(0.95f, 0.98f);
trt.offsetMin = trt.offsetMax = Vector2.zero;

_body = MakeText(panel.transform, "", 16);
_body.alignment = TextAnchor.UpperLeft;
var brt = _body.rectTransform;
brt.anchorMin = new Vector2(0.06f, 0.58f);
brt.anchorMax = new Vector2(0.94f, 0.9f);
brt.offsetMin = brt.offsetMax = Vector2.zero;

float cardW = 170f;
float gap = 18f;
float startX = -((cardW + gap));
for (int i = 0; i < 3; i++)
{
    var card = new GameObject("Card" + i, typeof(RectTransform), typeof(Image), typeof(Button));
    card.transform.SetParent(panel.transform, false);
    var crt = card.GetComponent<RectTransform>();
    crt.anchorMin = crt.anchorMax = new Vector2(0.5f, 0.5f);
    crt.sizeDelta = new Vector2(cardW, 210f);
    crt.anchoredPosition = new Vector2(startX + i * (cardW + gap), -40f);
    var img = card.GetComponent<Image>();
    img.color = new Color(0.28f, 0.22f, 0.18f, 1f);
    _cardBgs.Add(img);

    var iconGo = new GameObject("Icon", typeof(RectTransform), typeof(Image));
    iconGo.transform.SetParent(card.transform, false);
    var irt = iconGo.GetComponent<RectTransform>();
    irt.anchorMin = irt.anchorMax = new Vector2(0.5f, 1f);
    irt.pivot = new Vector2(0.5f, 1f);
    irt.anchoredPosition = new Vector2(0f, -8f);
    irt.sizeDelta = new Vector2(64f, 64f);
    var iconImg = iconGo.GetComponent<Image>();
    iconImg.color = Color.white;
    _cardIcons.Add(iconImg);
}

```

----- 第 14 页 -----

```

        var label = MakeText(card.transform, "", 15);
        label.alignment = TextAnchor.UpperCenter;
        var lrt = label.rectTransform;
        lrt.anchorMin = new Vector2(0.06f, 0.04f);
        lrt.anchorMax = new Vector2(0.94f, 0.62f);
        lrt.offsetMin = lrt.offsetMax = Vector2.zero;
        _cardLabels.Add(label);

        int idx = i;
        card.GetComponent<Button>().onClick.AddListener(() => Select(idx));
    }

    MakeBtn(panel.transform, "Confirm", "招募选中", new Vector2(-110f, 28f), OnConfirmRecruit);
    MakeBtn(panel.transform, "Reroll", "换一批", new Vector2(110f, 28f), () =>
    {
        RerollOffers();
        Refresh();
    });
    MakeBtn(panel.transform, "Close", "关闭", new Vector2(0f, -28f), Close);
}

void Close()
{
    if (_root != null) _root.SetActive(false);
}

static Text MakeText(Transform parent, string content, int size)
{
    var go = new GameObject("T", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    Stretch(go.GetComponent<RectTransform>());
    var t = go.GetComponent<Text>();
    t.text = content;
    t.fontSize = size;
    t.alignment = TextAnchor.MiddleCenter;
    t.color = Color.white;
    t.horizontalOverflow = HorizontalWrapMode.Wrap;
    t.verticalOverflow = VerticalWrapMode.Overflow;
    t.font = GameFonts.GetChinese();
    if (t.font == null) t.font = Resources.GetBuiltinResource<Font>("Arial.ttf");
    return t;
}

static void MakeBtn(Transform parent, string name, string label, Vector2 pos, UnityEngine.Events.UnityAction
onClick)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = go.GetComponent<RectTransform>();
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0f);
    rt.pivot = new Vector2(0.5f, 0f);

```

----- 第 15 页 -----

```

        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(160f, 44f);
        go.GetComponent<Image>().color = new Color(0.45f, 0.32f, 0.2f, 1f);
        var t = MakeText(go.transform, label, 20);
        Stretch(t.rectTransform);
        go.GetComponent<Button>().onClick.AddListener(onClick);
    }

    static void Stretch(RectTransform rt)
    {
        if (rt == null) return;
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
    }
}

// ===== FILE: Assets/Scripts/UI/TownHeroCostumePreview.cs =====
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// 城镇角色页：用与战斗相同的 wanjia SPUM + HeroCostumeManager 实时换装预览。
/// 不覆盖角色页预制体结构；在 Portrait 位挂 RawImage + 独立相机渲染。
/// </summary>
public class TownHeroCostumePreview : MonoBehaviour
{
    const string PrefabPath = "Units/wanjia";
    const int RtW = 256;
    const int RtH = 384;

    public RawImage targetImage;
    public RectTransform host;

    Camera _cam;
    RenderTexture _rt;
    GameObject _heroGo;
    HeroCostumeManager _costume;
    bool _built;

    public static TownHeroCostumePreview EnsureOn(CharacterUI ui)
    {
        if (ui == null) return null;
        var existing = ui.GetComponent<TownHeroCostumePreview>();
        if (existing != null)
        {
            existing.BindHost(ui);
            return existing;
        }
        var p = ui.gameObject.AddComponent<TownHeroCostumePreview>();
    }
}

```

----- 第 16 页 -----

```
p.BindHost(ui);
return p;
}

void BindHost(CharacterUI ui)
{
    if (ui.portraitImage != null)
        host = ui.portraitImage.rectTransform;
    if (host == null)
    {
        var t = ui.transform.Find("Content/Stage/Portrait");
        if (t != null) host = t as RectTransform;
    }
}

public void Show()
{
    BuildIfNeeded();
    if (_heroGo != null) _heroGo.SetActive(true);
    if (_cam != null) _cam.enabled = true;
    RefreshCostume();
    if (targetImage != null) targetImage.enabled = true;
    if (host != null)
    {
        var img = host.GetComponent<Image>();
        if (img != null) img.enabled = false; // 隐藏静态立绘，显示 SPUM
    }
}

public void Hide()
{
    if (_cam != null) _cam.enabled = false;
    if (_heroGo != null) _heroGo.SetActive(false);
    if (targetImage != null) targetImage.enabled = false;
    if (host != null)
    {
        var img = host.GetComponent<Image>();
        if (img != null) img.enabled = true;
    }
}

public void RefreshCostume()
{
    if (_costume != null)
        _costume.RefreshCostume();
}

void OnEnable()
{
    if (GridBackpackSystem.Instance != null)
```


----- 第 17 页 -----

```
        GridBackpackSystem.Instance.OnCostumeChanged += RefreshCostume;
    if (GridBackpackSystem.Instance != null)
        GridBackpackSystem.Instance.OnBackpackChanged += RefreshCostume;
}

void OnDisable()
{
    if (GridBackpackSystem.Instance != null)
    {
        GridBackpackSystem.Instance.OnCostumeChanged -= RefreshCostume;
        GridBackpackSystem.Instance.OnBackpackChanged -= RefreshCostume;
    }
}

void OnDestroy()
{
    if (_rt != null)
    {
        if (_cam != null) _cam.targetTexture = null;
        _rt.Release();
        Destroy(_rt);
        _rt = null;
    }
    if (_heroGo != null) Destroy(_heroGo);
    if (_cam != null) Destroy(_cam.gameObject);
}

void BuildIfNeeded()
{
    if (_built) return;
    if (host == null) return;

    // RawImage 盖在 Portrait 上
    if (targetImage == null)
    {
        var go = new GameObject("SpumPreview", typeof(RectTransform), typeof(RawImage));
        go.transform.SetParent(host, false);
        var rt = go.GetComponent<RectTransform>();
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        targetImage = go.GetComponent<RawImage>();
        targetImage.raycastTarget = false;
        targetImage.color = Color.white;
    }

    _rt = new RenderTexture(RtW, RtH, 16, RenderTextureFormat.ARGB32);
    _rt.Create();
    targetImage.texture = _rt;
}
```

----- 第 18 页 -----

```
var camGo = new GameObject("TownHeroPreviewCam");
DontDestroyOnLoad(camGo);
_cam = camGo.AddComponent<Camera>();
_cam.orthographic = true;
_cam.orthographicSize = 1.15f;
_cam.clearFlags = CameraClearFlags.SolidColor;
_cam.backgroundColor = new Color(0f, 0f, 0f, 0f);
_cam.allowHDR = false;
_cam.allowMSAA = false;
_cam.cullingMask = ~0;
_cam.targetTexture = _rt;
_cam.nearClipPlane = 0.1f;
_cam.farClipPlane = 20f;
_cam.depth = -80;
_cam.enabled = false;
camGo.transform.position = new Vector3(4800f, 4800f, -10f);

var prefab = Resources.Load<GameObject>(PrefabPath);
if (prefab == null)
{
    Debug.LogWarning("[TownHeroCostumePreview] 未找到 Units/wanjia");
    _built = true;
    return;
}

_heroGo = Instantiate(prefab);
_heroGo.name = "TownHeroPreview";
DontDestroyOnLoad(_heroGo);
_heroGo.transform.position = camGo.transform.position + new Vector3(0f, -0.2f, 8f);
_heroGo.transform.localScale = Vector3.one * 1.2f;
_heroGo.transform.rotation = Quaternion.identity;

// 关掉战斗组件, 只留形象
foreach (var rb in _heroGo.GetComponentsInChildren<Rigidbody2D>(true))
{
    rb.simulated = false;
    rb.velocity = Vector2.zero;
}

var hero = _heroGo.GetComponent<Hero>();
if (hero != null) hero.enabled = false;
foreach (var ub in _heroGo.GetComponentsInChildren<UnitBase>(true))
    ub.enabled = false;

_costume = _heroGo.GetComponent<HeroCostumeManager>();
if (_costume == null) _costume = _heroGo.AddComponent<HeroCostumeManager>();
if (_costume.spumPrefabs == null)
    _costume.spumPrefabs = _heroGo.GetComponentInChildren<SPUM_Prefabs>(true);
if (_costume.spriteList == null)
    _costume.spriteList = _heroGo.GetComponentInChildren<SPUM_SpriteList>(true);
// 城镇预览不抢占战斗侧 Instance, 避免回城/开战互相覆盖
```

----- 第 19 页 -----

```

        if (HeroCostumeManager.Instance == _costume)
            HeroCostumeManager.Instance = null;

        // 分层：预览单位进专属层，相机只看这一层，避免扫到城镇 UI
        int layer = LayerMask.NameToLayer("Default");
        SetLayerRecursive(_heroGo, layer);

        _built = true;
        RefreshCostume();
    }

    static void SetLayerRecursive(GameObject go, int layer)
    {
        if (go == null) return;
        go.layer = layer;
        for (int i = 0; i < go.transform.childCount; i++)
            SetLayerRecursive(go.transform.GetChild(i).gameObject, layer);
    }
}

// ===== FILE: Assets/Scripts/Unit/HeroCostumeManager.cs =====
using UnityEngine;
using System.Collections.Generic;

/// <summary>
/// 英雄换装管理器：监听装备变化→自动更新玩家SPUM形象
///
/// 核心对应关系（装备槽 → SPUM SpriteList属性 → SPUM资源文件夹）：
/// 头(Head)   → _hairList   → 4_Helmet / 0_Hair / 2_Hair
/// 胸(Chest)  → _armorList  → 5_Armor
/// 手(Hands)  → _clothList  → 2_Cloth
/// 脚(Feet)   → _pantList   → 3_Pant
/// 披风(Cape) → _backList   → 7_Back / 10_Back
/// 武器(Weapons) → _weaponList → 6_Weapons/{子文件夹}
///
/// 效率优化：启动时扫描一次全部SPUM资源，建立 spumName→路径 缓存字典
/// 之后换装直接查字典，避免每次 Resources.LoadAll 遍历
/// </summary>
public class HeroCostumeManager : MonoBehaviour
{
    public static HeroCostumeManager Instance;

    [Header("SPUM组件引用")]
    public SPUM_Prefabs spumPrefabs;
    public SPUM_SpriteList spriteList;

    // SPUM Resources 基础路径（相对于 Assets/SPUM/Resources/）
    private const string SPUM_SPRITE_BASE = "Addons/{0}/0_Unit/0_Sprite";

    /// <summary>

```

----- 第 20 页 -----

```

/// 三个Addon版本（按优先级搜索：Legacy资源最多，优先）
/// </summary>
private static readonly string[] AddonVersions = { "Legacy", "Ver121", "Ver300" };

/// <summary>
/// 槽位 → SPUM SpriteList属性名（用于GetPartList获取SpriteRenderer列表）
/// 注意：Head用"Hair"因为SPUM没有_helmetList，头盔通过hairList显示
/// </summary>
private static readonly Dictionary<EquipSlotType, string> SlotToPartType = new Dictionary<EquipSlotType, string>
{
    { EquipSlotType.Head, "Hair" },
    { EquipSlotType.Chest, "Armor" },
    { EquipSlotType.Hands, "Cloth" },
    { EquipSlotType.Feet, "Pant" },
    { EquipSlotType.Cape, "Back" },
    { EquipSlotType.MainHand, "Weapons" },
    { EquipSlotType.OffHand, "Weapons" }
};

/// <summary>
/// 槽位 → 所有可能的SPUM资源文件夹名称（不同版本文件夹名可能不同）
/// Head: Legacy/Ver121用4_Helmet和0_Hair, Ver300用2_Hair
/// Cape: Legacy用7_Back, Ver300用10_Back
/// </summary>
private static readonly Dictionary<EquipSlotType, string[]> SlotToAllFolders = new Dictionary<EquipSlotType,
string[]>
{
    { EquipSlotType.Head, new[] { "4_Helmet", "0_Hair", "2_Hair" } },
    { EquipSlotType.Chest, new[] { "5_Armor" } },
    { EquipSlotType.Hands, new[] { "2_Cloth" } },
    { EquipSlotType.Feet, new[] { "3_Pant" } },
    { EquipSlotType.Cape, new[] { "7_Back", "10_Back" } },
    { EquipSlotType.MainHand, new[] { "6_Weapons" } },
    { EquipSlotType.OffHand, new[] { "6_Weapons" } }
};

/// <summary>
/// 所有可能的武器子文件夹（Legacy和Ver121的子文件夹不同）
/// Legacy: 0_Sword, 1_Axe, 2_Bow, 3_Shield, 4_Spear, 5_Wand, 6_Hammer
/// Ver121: 2_Axe, 7_Shield
/// </summary>
private static readonly string[] AllWeaponSubFolders =
{
    "0_Sword", "1_Axe", "2_Bow", "3_Shield", "4_Spear", "5_Wand", "6_Hammer",
    "2_Axe", "7_Shield"
};

/// <summary>
/// 多切片sprite的子精灵名称（Cloth/Armor可能包含Body/Left/Right三个切片）
/// </summary>
private static readonly string[] MultiSliceNames = { "Body", "Left", "Right" };

```

----- 第 21 页 -----

```
/// <summary>
/// 缓存字典: spumName → 完整Resources路径
/// 启动时扫描一次, 之后换装直接查字典, O(1)复杂度
/// </summary>
private Dictionary<string, string> _spumPathCache = new Dictionary<string, string>();

/// <summary>
/// 缓存字典: 路径 → 已加载的Sprite数组 (避免重复Resources.LoadAll)
/// </summary>
private Dictionary<string, Sprite[]> _spriteCache = new Dictionary<string, Sprite[]>();

/// <summary>
/// 缓存是否已初始化
/// </summary>
private bool _cacheInitialized = false;

void Awake()
{
    Instance = this;

    if (spumPrefabs == null)
        spumPrefabs = GetComponent<SPUM_Prefabs>();

    if (spriteList == null)
        spriteList = GetComponent<SPUM_SpriteList>();

    if (spriteList == null)
        spriteList = GetComponentInChildren<SPUM_SpriteList>();
}

void OnEnable()
{
    EnsureSubscribed();
}

void Start()
{
    BuildPathCache();
    EnsureSubscribed();
    // 进场时按当前穿戴刷一次, 避免只改了数值没改外观
    RefreshCostume();
}

void OnDisable()
{
    Unsubscribe();
}

void OnDestroy()
```

----- 第 22 页 -----

```
{
    Unsubscribe();
    if (Instance == this) Instance = null;
}

bool _subscribed;

void EnsureSubscribed()
{
    if (_subscribed) return;
    if (GridBackpackSystem.Instance == null) return;
    GridBackpackSystem.Instance.OnCostumeChanged += RefreshCostume;
    _subscribed = true;
}

void Unsubscribe()
{
    if (!_subscribed) return;
    if (GridBackpackSystem.Instance != null)
        GridBackpackSystem.Instance.OnCostumeChanged -= RefreshCostume;
    _subscribed = false;
}

/// <summary>
/// 启动时扫描全部SPUM资源，建立 spumName→路径 缓存字典
/// 只执行一次，之后所有换装都查字典
/// </summary>
private void BuildPathCache()
{
    if (_cacheInitialized) return;
    _cacheInitialized = true;

    Debug.Log("[HeroCostumeManager] 开始扫描SPUM资源，建立路径缓存...");

    int count = 0;

    foreach (string version in AddonVersions)
    {
        string basePath = string.Format(SPUM_SPRITE_BASE, version);

        foreach (var kvp in SlotToAllFolders)
        {
            EquipSlotType slot = kvp.Key;
            string[] folders = kvp.Value;
            bool isWeapon = (slot == EquipSlotType.MainHand || slot == EquipSlotType.OffHand);

            foreach (string folder in folders)
            {
                if (isWeapon)
                {
```

----- 第 23 页 -----

```

        // 武器：遍历所有子文件夹
        foreach (string subFolder in AllWeaponSubFolders)
        {
            string path = $"{basePath}/{folder}/{subFolder}";
            count += ScanAndCacheDirectory(path, subFolder);
        }
    }
    else
    {
        // 非武器：直接扫描文件夹
        count += ScanAndCacheDirectory($"{basePath}/{folder}", null);
    }
}

}

Debug.Log($"{HeroCostumeManager} 路径缓存建立完成，共缓存 {count} 个SPUM精灵图");
}

/// <summary>
/// 扫描单个Resources目录，将文件名→路径存入缓存
/// 使用 Resources.LoadAll 加载目录下所有Sprite
/// </summary>
private int ScanAndCacheDirectory(string resourcePath, string context)
{
    // 加载该目录下所有Sprite
    Sprite[] sprites = Resources.LoadAll<Sprite>(resourcePath);
    if (sprites == null || sprites.Length == 0) return 0;

    int count = 0;
    // 收集该目录下所有不重复的文件名（去掉切片后缀）
    HashSet<string> fileNames = new HashSet<string>();

    foreach (Sprite sp in sprites)
    {
        // sprite.name 可能是 "Helmet_1" 或 "Body"/"Left"/"Right"（多切片）
        // 多切片的父文件名需要从sprite名推导
        string fileName = sp.name;

        // 多切片的子精灵（Body/Left/Right）跳过，它们属于父文件
        if (System.Array.IndexOf(MultiSliceNames, fileName) >= 0) continue;

        if (!fileNames.Contains(fileName))
        {
            fileNames.Add(fileName);
            // 文件名 → 完整Resources路径（不含扩展名）
            string fullPath = $"{resourcePath}/{fileName}";
            if (!_spumPathCache.ContainsKey(fileName))
            {
                _spumPathCache[fileName] = fullPath;
            }
        }
    }
}

```

----- 第 24 页 -----

```
        count++;
    }
}

// 缓存该目录的Sprite数组（供后续直接使用）
if (!_spriteCache.ContainsKey(resourcePath))
{
    _spriteCache[resourcePath] = sprites;
}

return count;
}

/// <summary>
/// 刷新全部换装：遍历所有装备槽，根据spumName加载SPUM精灵图并替换
/// 使用缓存字典，O(1)查找路径，O(1)查找已加载的Sprite
/// </summary>
public void RefreshCostume()
{
    if (spriteList == null)
    {
        Debug.LogWarning("[HeroCostumeManager] spriteList 未赋值，跳过换装");
        return;
    }
    if (GridBackpackSystem.Instance == null) return;

    // 确保缓存已初始化
    if (!_cacheInitialized) BuildPathCache();

    // 固定顺序：先主手后副手，避免字典遍历顺序不确定
    EquipSlotType[] order =
    {
        EquipSlotType.Head, EquipSlotType.Chest, EquipSlotType.Hands,
        EquipSlotType.Feet, EquipSlotType.Cape, EquipSlotType.MainHand, EquipSlotType.OffHand
    };

    for (int i = 0; i < order.Length; i++)
    {
        EquipSlotType slot = order[i];
        if (!SlotToPartType.TryGetValue(slot, out string partType)) continue;

        EquipInstance equip = GridBackpackSystem.Instance.GetEquippedInSlot(slot);
        if (equip != null && equip.template != null && !string.IsNullOrEmpty(equip.template.spumName))
        {
            ApplySpriteToPart(partType, slot, equip.template.spumName);
        }
    }

    Debug.Log("[HeroCostumeManager] 换装刷新完成");
}
```


----- 第 25 页 -----

```
}

/// <summary>
/// 加载SPUM精灵图并应用到指定部件
/// 自动处理单切片和多切片（Body/Left/Right）情况
/// 使用缓存，避免重复 Resources.LoadAll
/// </summary>
private void ApplySpriteToPart(string partType, EquipSlotType slot, string spumName)
{
    if (string.IsNullOrEmpty(spumName)) return;

    List<SpriteRenderer> targetList = GetPartList(partType);
    if (targetList == null || targetList.Count == 0)
    {
        Debug.LogWarning($"[HeroCostumeManager] {partType} 的 SpriteRenderer 列表为空");
        return;
    }

    // 从缓存获取Sprite数组
    Sprite[] sprites = GetCachedSprites(spumName);
    if (sprites == null || sprites.Length == 0)
    {
        Debug.LogWarning($"[HeroCostumeManager] 未找到精灵: slot={slot}, spumName={spumName}");
        return;
    }

    if (sprites.Length == 1)
    {
        // 单切片：设置列表中所有SpriteRenderer
        foreach (var sr in targetList)
        {
            if (sr != null) sr.sprite = sprites[0];
        }
    }
    else
    {
        // 多切片：按名称匹配 Body/Left/Right → 对应索引 0/1/2
        for (int i = 0; i < targetList.Count && i < MultiSliceNames.Length; i++)
        {
            if (targetList[i] == null) continue;
            Sprite sub = System.Array.Find(sprites, s => s.name == MultiSliceNames[i]);
            if (sub != null)
            {
                targetList[i].sprite = sub;
            }
        }
    }

    // 更新路径字符串，防止ResyncData覆盖
    UpdatePathString(partType, spumName);
}
```

----- 第 26 页 -----

```

        Debug.Log($"[HeroCostumeManager] 换装成功: {partType} ← {spumName} ({sprites.Length} 切片)");
    }

    /// <summary>
    /// 从缓存获取Sprite数组
    /// 1. 先查 _spumPathCache 获取路径 (0(1))
    /// 2. 再查 _spriteCache 获取已加载的Sprite (0(1))
    /// 3. 若sprite未缓存, 按路径加载一次并存入缓存
    /// </summary>
    private Sprite[] GetCachedSprites(string spumName)
    {
        // 1. 查路径缓存
        if (!_spumPathCache.TryGetValue(spumName, out string fullPath))
        {
            // 缓存未命中: 可能扫描时遗漏, 尝试直接加载
            return FallbackLoad(spumName);
        }

        // 2. 查sprite缓存 (按目录路径)
        // fullPath 格式: "Addons/Legacy/0_Unit/0_Sprite/4_Helmet/Helmet_1"
        // 目录路径 = 去掉最后的文件名
        int lastSlash = fullPath.LastIndexOf('/');
        if (lastSlash <= 0) return FallbackLoad(spumName);

        string dirPath = fullPath.Substring(0, lastSlash);

        // 3. 目录的Sprite数组已缓存就直接用
        if (_spriteCache.TryGetValue(dirPath, out Sprite[] cached)
        {
            // 从目录的所有Sprite中筛选属于该文件的切片
            return FilterSpritesByFile(cached, spumName);
        }

        // 4. 目录未缓存: 加载一次
        Sprite[] sprites = Resources.LoadAll<Sprite>(dirPath);
        if (sprites != null && sprites.Length > 0)
        {
            _spriteCache[dirPath] = sprites;
            return FilterSpritesByFile(sprites, spumName);
        }

        return null;
    }

    /// <summary>
    /// 从目录的所有Sprite中筛选属于指定文件的切片
    /// 单切片: 返回文件名相同的Sprite
    /// 多切片: 返回 Body/Left/Right 三个切片
    /// </summary>

```

----- 第 27 页 -----

```
private Sprite[] FilterSpritesByFile(Sprite[] allSprites, string fileName)
{
    if (allSprites == null || allSprites.Length == 0) return null;

    // 先找和文件名完全相同的Sprite (单切片情况)
    foreach (Sprite sp in allSprites)
    {
        if (sp.name == fileName) return new Sprite[] { sp };
    }

    // 没找到同名, 说明是多切片, 收集 Body/Left/Right
    List<Sprite> result = new List<Sprite>();
    foreach (string sliceName in MultiSliceNames)
    {
        Sprite sub = System.Array.Find(allSprites, s => s.name == sliceName);
        if (sub != null) result.Add(sub);
    }

    return result.Count > 0 ? result.ToArray() : null;
}

/// <summary>
/// 缓存未命中时的兜底加载
/// 直接用spumName作为路径尝试加载
/// </summary>
private Sprite[] FallbackLoad(string spumName)
{
    Sprite[] sprites = Resources.LoadAll<Sprite>(spumName);
    if (sprites != null && sprites.Length > 0)
    {
        _spumPathCache[spumName] = spumName;
        return sprites;
    }
    return null;
}

/// <summary>
/// 获取部件对应的SpriteRenderer列表
/// </summary>
private List<SpriteRenderer> GetPartList(string partType)
{
    if (spriteList == null) return null;

    switch (partType)
    {
        case "Hair":    return spriteList._hairList;
        case "Cloth":   return spriteList._clothList;
        case "Armor":   return spriteList._armorList;
        case "Pant":    return spriteList._pantList;
        case "Back":    return spriteList._backList;
```

```
----- 第 28 页 -----
        case "Weapons": return spriteList._weaponList;
        default: return null;
    }
}

/// <summary>
/// 更新路径字符串列表，防止SPUM的ResyncData用旧路径覆盖换装结果
/// 直接从缓存字典获取路径
/// </summary>
private void UpdatePathString(string partType, string spumName)
{
    if (spriteList == null) return;

    // 从缓存获取实际路径
    string path;
    if (!_spumPathCache.TryGetValue(spumName, out path))
    {
        return; // 缓存中没有，无法更新
    }

    switch (partType)
    {
        case "Hair":
            SetPathStringList(spriteList._hairListString, path);
            break;
        case "Cloth":
            SetPathStringList(spriteList._clothListString, path);
            break;
        case "Armor":
            SetPathStringList(spriteList._armorListString, path);
            break;
        case "Pant":
            SetPathStringList(spriteList._pantListString, path);
            break;
        case "Back":
            SetPathStringList(spriteList._backListString, path);
            break;
        case "Weapons":
            SetPathStringList(spriteList._weaponListString, path);
            break;
    }
}

/// <summary>
/// 安全设置路径字符串列表的第一个元素
/// </summary>
private void SetPathStringList(List<string> list, string path)
{
    if (list == null) return;
    if (list.Count == 0) list.Add(path);
}
```

----- 第 29 页 -----

```

        else list[0] = path;
    }
}

// ===== FILE: Assets/Scripts/Systems/MercenaryManager.cs =====
using UnityEngine;
using System.Collections.Generic;

/// <summary>
/// 佣兵管理器：头像查询、预制体加载、出战生成与回收。
/// 生成方式与 Hero 对齐：挂到 unit → 写世界坐标 → Init 朝向；不改大小/排序。
/// </summary>
public class MercenaryManager : Singleton<MercenaryManager>
{
    [Header("角色注册表")]
    public CharacterRegistry registry;

    private List<Mercenary> _activeMercs = new List<Mercenary>();

    public const string PLAYER_ID = "wanjia";

    protected override void Awake()
    {
        base.Awake();
        if (registry == null)
        {
            registry = Resources.Load<CharacterRegistry>("Config/CharacterRegistry");
            if (registry == null)
                Debug.LogWarning("[MercenaryManager] 未找到CharacterRegistry，请运行 Tools/生成角色注册表 菜单生成");
        }
    }

    #region 头像/配置查询

    public Sprite GetIcon(string characterId)
    {
        Sprite sp = registry != null ? registry.GetIcon(characterId) : null;
        if (sp != null) return sp;
        if (string.IsNullOrEmpty(characterId)) return null;
        sp = Resources.Load<Sprite>("UI/Heads/icon_" + characterId);
        if (sp != null) return sp;
        // 101/102 盾兵共用头像
        if (characterId.StartsWith("dunbing"))
            return Resources.Load<Sprite>("UI/Heads/icon_dunbing102");
        return null;
    }

    public string GetJobName(string characterId)
    {
        return registry != null ? registry.GetJobName(characterId) : characterId;
    }
}

```

----- 第 30 页 -----

```

    }

    public string GetPrefabName(string characterId)
    {
        return registry != null ? registry.GetPrefabName(characterId) : characterId;
    }

    static GameObject LoadUnitPrefab(string prefabName, string mercId)
    {
        if (!string.IsNullOrEmpty(prefabName))
        {
            var p = Resources.Load<GameObject>("Units/" + prefabName);
            if (p != null) return p;
        }
        if (!string.IsNullOrEmpty(mercId) && mercId != prefabName)
        {
            var p = Resources.Load<GameObject>("Units/" + mercId);
            if (p != null) return p;
        }
        if (!string.IsNullOrEmpty(mercId) && mercId.Length >= 3)
        {
            char last = mercId[mercId.Length - 1];
            if (last == '2' || last == '3' || last == '4')
            {
                string fallback = mercId.Substring(0, mercId.Length - 1) + "1";
                var p = Resources.Load<GameObject>("Units/" + fallback);
                if (p != null)
                {
                    Debug.LogWarning($"[MercenaryManager] {mercId} 无预制体，回退 {fallback}");
                    return p;
                }
            }
        }
        return null;
    }

    public Sprite GetPlayerIcon() => GetIcon(PLAYER_ID);

#endregion

#region 出战佣兵

    public int GetMaxMercSlots()
    {
        int tavernLevel = SaveSystem.Instance != null && SaveSystem.Instance.Data != null
            ? SaveSystem.Instance.Data.townLevel.tavern
            : 0;
        return Mathf.Clamp(tavernLevel, 0, 2);
    }

```

----- 第 31 页 -----

```

try
{
    var hero = Hero.Instance;
    if (hero != null && hero.attr != null)
    {
        hp = hero.attr.GetAttr(AttrType.MaxHp);
        atk = hero.attr.GetAttr(AttrType.Attack);
        def = hero.attr.GetAttr(AttrType.Defense);
        spd = hero.attr.GetAttr(AttrType.AttackSpeed);
        crit = hero.attr.GetAttr(AttrType.CritRate);
    }
    else
    {
        // 城镇无 Hero 时用天赋汇总估算展示
        int leftN = TalentDefs.LeftUnlockedCount(SaveSystem.Instance?.Data?.talents);
        for (int i = 0; i < leftN; i++)
        {
            var e = TalentDefs.Left[i].effect;
            switch (e.kind)
            {
                case TalentDefs.AttrKind.Attack: atk += e.value; break;
                case TalentDefs.AttrKind.Hp: hp += e.value; break;
                case TalentDefs.AttrKind.Defense: def += e.value; break;
                case TalentDefs.AttrKind.CritRate: crit += e.value; break;
                case TalentDefs.AttrKind.AtkSpeed: spd += e.value; break;
            }
        }
    }
}
catch { }

float critDisplay = crit;
if (Hero.Instance != null && Hero.Instance.attr != null && critDisplay <= 1.5f)
    critDisplay *= 100f;

if (attrHpText != null) attrHpText.text = Mathf.RoundToInt(hp).ToString("N0");
if (attrAtkText != null) attrAtkText.text = Mathf.RoundToInt(atk).ToString("N0");
if (attrDefText != null) attrDefText.text = Mathf.RoundToInt(def).ToString("N0");
if (attrSpdText != null) attrSpdText.text = spd.ToString("0.##");
if (attrCritText != null) attrCritText.text = critDisplay.ToString("0.#") + "%";
if (attrResistText != null) attrResistText.text = resist.ToString("0.#") + "%";
}

void RefreshBag()
{
    backpackGrid?.Refresh();
    int unlocked = backpackGrid != null ? backpackGrid.UnlockedSlotCount() : GameConfig.BACKPACK_DEFAULT_ROWS *
GameConfig.BACKPACK_WIDTH;
    int used = 0;
    if (backpackGrid != null)
    {

```

----- 第 32 页 -----

```

        for (int i = 0; i < backpackGrid.cells.Count; i++)
            if (backpackGrid.cells[i]?.equippedItem != null) used++;
    }
    if (bagCapacityText != null)
        bagCapacityText.text = used + "/" + unlocked;
}

void WireClicks()
{
    if (_wired) return;
    _wired = true;
    if (talentButton != null)
    {
        talentButton.onClick.RemoveAllListeners();
        talentButton.onClick.AddListener(OpenTalent);
    }
    if (skillButton != null)
    {
        skillButton.onClick.RemoveAllListeners();
        skillButton.onClick.AddListener(OpenSkillSelect);
    }
    if (leftSkillButton != null)
    {
        leftSkillButton.onClick.RemoveAllListeners();
        leftSkillButton.onClick.AddListener(OpenSkillSelect);
    }
    if (flipPortraitButton != null)
    {
        flipPortraitButton.onClick.RemoveAllListeners();
        flipPortraitButton.onClick.AddListener(FlipPortrait);
    }
    if (bagExpandButton != null)
    {
        bagExpandButton.onClick.RemoveAllListeners();
        bagExpandButton.onClick.AddListener(() =>
            UIManager.Instance?.ShowToast("扩容请在天赋中解锁背包行"));
    }
}

public void OpenTalent()
{
    if (TalentUI.Instance != null)
    {
        TalentUI.Instance.Show();
        return;
    }
    var prefab = Resources.Load<GameObject>("Prefabs/Talent/TalentUI");
    if (prefab == null)
    {
        UIManager.Instance?.ShowToast("天赋界面未就绪");
    }
}

```


----- 第 33 页 -----

```
        return;
    }
    var go = Instantiate(prefab);
    go.name = "TalentUI";
    go.GetComponent<TalentUI>()?.Show();
}

public void OpenSkillSelect()
{
    if (skillSelect == null)
        skillSelect = GetComponentInChildren<SkillSelectUI>(true);
    if (skillSelect == null)
    {
        UIManager.Instance?.ShowToast("技能界面未就绪");
        return;
    }
    skillSelect.Show();
}

void FlipPortrait()
{
    if (portraitImage == null) return;
    _portraitFlipped = !_portraitFlipped;
    var s = portraitImage.rectTransform.localScale;
    float ax = Mathf.Abs(s.x) < 0.01f ? 1f : Mathf.Abs(s.x);
    s.x = ax * (_portraitFlipped ? -1f : 1f);
    portraitImage.rectTransform.localScale = s;
}

/// <summary>立绘用自身像素大小，仅翻转</summary>
public void SetPortrait(Sprite sprite, bool flip = false)
{
    if (portraitImage == null) return;
    portraitImage.sprite = sprite;
    portraitImage.preserveAspect = true;
    if (sprite != null)
    {
        portraitImage.enabled = true;
        portraitImage.SetNativeSize();
    }
    _portraitFlipped = flip;
    var s = portraitImage.rectTransform.localScale;
    float ax = Mathf.Abs(s.x) < 0.01f ? 1f : Mathf.Abs(s.x);
    s.x = ax * (flip ? -1f : 1f);
    portraitImage.rectTransform.localScale = s;
}

void StripLocalChrome()
{
    DestroyChildNamed(transform, "TopBar");
}
```

----- 第 34 页 -----

```
        DestroyChildNamed(transform, "SharedResourceBar");
        DestroyChildNamed(transform, "BottomNav");
        DestroyChildNamed(transform, "MainBottomNav");
    }

    static void DestroyChildNamed(Transform root, string name)
    {
        var t = root.Find(name);
        if (t != null) Destroy(t.gameObject);
    }

    void EnsureVisibleTransform()
    {
        if (transform.localScale.sqrMagnitude < 0.0001f)
            transform.localScale = Vector3.one;
    }

    bool _canvasConfigured;
    void ConfigureHostCanvasOnce()
    {
        if (_canvasConfigured) return;
        EnsureVisibleTransform();
        var hall = GetComponentInParent<GuildHallUI>();
        bool nested = hall != null && hall.gameObject != gameObject;
        if (nested)
        {
            var ray = GetComponent<GraphicRaycaster>();
            if (ray != null) Destroy(ray);
            var scaler = GetComponent<CanvasScaler>();
            if (scaler != null) Destroy(scaler);
            var own = GetComponent<Canvas>();
            if (own != null) Destroy(own);
            _canvasConfigured = true;
            return;
        }
        var canvas = GetComponent<Canvas>();
        if (canvas == null) canvas = gameObject.AddComponent<Canvas>();
        canvas.enabled = true;
        canvas.overrideSorting = true;
        canvas.sortingOrder = 20;
        UICanvasSetup.Apply(canvas, Camera.main);
        if (GetComponent<GraphicRaycaster>() == null)
            gameObject.AddComponent<GraphicRaycaster>();
        _canvasConfigured = true;
    }

    static void SetGuildOverlay(bool characterOpen)
    {
        TavernUI.SetGuildHallOverlayMode(characterOpen);
    }
```

----- 第 35 页 -----

```

public void AutoBind()
{
    titleText = FindTxt("Content/Header/TitleText");
    talentButton = transform.Find("Content/Stage/RightButtons/TalentButton").GetComponent<Button>();
    skillButton = transform.Find("Content/Stage/RightButtons/SkillButton").GetComponent<Button>();
    leftSkillButton = transform.Find("Content/Stage/LeftSkillButton").GetComponent<Button>();
    portraitImage = transform.Find("Content/Stage/Portrait").GetComponent<Image>();
    flipPortraitButton = transform.Find("Content/Stage/FlipButton").GetComponent<Button>();

    attrHpText = FindTxt("Content/AttrPanel/AttrHpRoot/AttrHp") ?? FindTxt("Content/AttrPanel/AttrHp");
    attrAtkText = FindTxt("Content/AttrPanel/AttrAtkRoot/AttrAtk") ?? FindTxt("Content/AttrPanel/AttrAtk");
    attrDefText = FindTxt("Content/AttrPanel/AttrDefRoot/AttrDef") ?? FindTxt("Content/AttrPanel/AttrDef");
    attrSpdText = FindTxt("Content/AttrPanel/AttrSpdRoot/AttrSpd") ?? FindTxt("Content/AttrPanel/AttrSpd");
    attrCritText = FindTxt("Content/AttrPanel/AttrCritRoot/AttrCrit") ?? FindTxt("Content/AttrPanel/AttrCrit");
    attrResistText = FindTxt("Content/AttrPanel/AttrResistRoot/AttrResist") ??
FindTxt("Content/AttrPanel/AttrResist");

    bagCapacityText = FindTxt("Content/BackpackPanel/CapacityText")
        ?? FindTxt("Content/BagPanel/CapacityText");
    bagExpandButton = transform.Find("Content/BackpackPanel/CapacityPlus").GetComponent<Button>()
        ?? transform.Find("Content/BagPanel/CapacityPlus").GetComponent<Button>();
    backpackGrid = GetComponent<TownBackpackGrid>();
    if (backpackGrid == null) backpackGrid = gameObject.AddComponent<TownBackpackGrid>();
    backpackGrid.BindFromHierarchy(transform);

    if (skillSelect == null)
        skillSelect = GetComponentInChildren<SkillSelectUI>(true);
}

Text FindTxt(string path)
{
    var t = transform.Find(path);
    return t != null ? t.GetComponent<Text>() : null;
}

/// <summary>编辑器首次建树；已换美术勿覆盖</summary>
public void BuildHierarchyForPrefab()
{
    for (int i = transform.childCount - 1; i >= 0; i--)
        DestroyImmediate(transform.GetChild(i).gameObject);

    var content = CreateRect(transform, "Content");
    var crt = content.GetComponent<RectTransform>();
    crt.anchorMin = Vector2.zero;
    crt.anchorMax = Vector2.one;
    crt.offsetMin = new Vector2(0f, BottomReserve);
    crt.offsetMax = new Vector2(0f, -TopReserve);

    // Header
    var header = CreateImg(content.transform, "Header", new Color(0.28f, 0.18f, 0.1f, 0.9f));

```

----- 第 36 页 -----

```

Set(header.rectTransform, 0f, 1f, 1f, 1f, 0.5f, 1f, 0f, 0f, 0f, 56f);
var title = CreateTxt(header.transform, "TitleText", "角色", 32, new Color(1f, 0.9f, 0.55f));
Set(title.rectTransform, 0f, 0f, 0.4f, 1f, 0f, 0.5f, 24f, 0f, 0f, 0f);
title.alignment = TextAnchor.MiddleLeft;

// Stage: 立绘 + 左技能钮 + 右天赋/技能 (无装备栏)
var stage = CreateImg(content.transform, "Stage", new Color(0.35f, 0.28f, 0.22f, 1f));
var srt = stage.rectTransform;
srt.anchorMin = new Vector2(0f, 0.42f);
srt.anchorMax = new Vector2(1f, 1f);
srt.offsetMin = new Vector2(12f, 0f);
srt.offsetMax = new Vector2(-12f, -64f);

var portrait = CreateImg(stage.transform, "Portrait", new Color(0.45f, 0.5f, 0.55f, 0.85f));
portrait.preserveAspect = true;
Set(portrait.rectTransform, 0.5f, 0.15f, 0.5f, 0.15f, 0.5f, 0f, 0f, 0f, 160f, 220f);

var leftBtn = CreateImg(stage.transform, "LeftSkillButton", new Color(0.4f, 0.32f, 0.55f, 1f));
Set(leftBtn.rectTransform, 0f, 0.55f, 0f, 0.55f, 0f, 0.5f, 16f, 0f, 100f, 72f);
leftBtn.gameObject.AddComponent<Button>().targetGraphic = leftBtn;
var leftLab = CreateTxt(leftBtn.transform, "Label", "技能", 24, Color.white);
Stretch(leftLab.rectTransform);

var right = CreateRect(stage.transform, "RightButtons");
Set(right.GetComponent<RectTransform>(), 1f, 0.2f, 1f, 0.95f, 1f, 1f, -12f, 0f, 120f, 0f);
BuildSideBtn(right.transform, "TalentButton", "天赋", 0f, new Color(0.35f, 0.55f, 0.35f, 1f));
BuildSideBtn(right.transform, "SkillButton", "技能", -90f, new Color(0.4f, 0.35f, 0.65f, 1f));

// Attr panel — 宽度随界面
var attrs = CreateImg(content.transform, "AttrPanel", new Color(0.22f, 0.16f, 0.12f, 0.95f));
var art = attrs.rectTransform;
art.anchorMin = new Vector2(0f, 0.30f);
art.anchorMax = new Vector2(1f, 0.42f);
art.offsetMin = new Vector2(12f, 4f);
art.offsetMax = new Vector2(-12f, -4f);
var attrTitle = CreateTxt(attrs.transform, "AttrTitle", "基础属性", 22, new Color(1f, 0.92f, 0.75f));
Set(attrTitle.rectTransform, 0.5f, 1f, 0.5f, 1f, 0.5f, 1f, 0f, -4f, 200f, 28f);
CreateAttr(attrs.transform, "AttrHp", "生命", "5240", 0f);
CreateAttr(attrs.transform, "AttrAtk", "攻击", "1280", 1f);
CreateAttr(attrs.transform, "AttrDef", "防御", "860", 2f);
CreateAttr(attrs.transform, "AttrSpd", "速度", "105", 3f);
CreateAttr(attrs.transform, "AttrCrit", "暴击", "18%", 4f);
CreateAttr(attrs.transform, "AttrResist", "抗性", "15%", 5f);

// Bag — 宽度自适应, 格子与战斗一致
var bag = CreateImg(content.transform, "BagPanel", new Color(0.4f, 0.28f, 0.16f, 1f));
var brt = bag.rectTransform;
brt.anchorMin = new Vector2(0f, 0f);
brt.anchorMax = new Vector2(1f, 0.30f);
brt.offsetMin = new Vector2(12f, 8f);

```

----- 第 37 页 -----

```

    brt.offsetMax = new Vector2(-12f, -4f);
    var bagTitle = CreateTxt(bag.transform, "BagTitle", "背包", 24, new Color(1f, 0.92f, 0.75f));
    Set(bagTitle.rectTransform, 0f, 1f, 0f, 1f, 0f, 1f, 16f, -8f, 100f, 32f);
    bagTitle.alignment = TextAnchor.MiddleLeft;
    var cap = CreateTxt(bag.transform, "CapacityText", "0/21", 22, new Color(1f, 0.95f, 0.8f));
    Set(cap.rectTransform, 1f, 1f, 1f, 1f, 1f, 1f, -56f, -8f, 100f, 32f);
    cap.alignment = TextAnchor.MiddleRight;
    var plus = CreateImg(bag.transform, "CapacityPlus", new Color(0.55f, 0.35f, 0.2f, 1f));
    Set(plus.rectTransform, 1f, 1f, 1f, 1f, 1f, 1f, -12f, -8f, 32f, 32f);
    plus.gameObject.AddComponent<Button>().targetGraphic = plus;
    var plusL = CreateTxt(plus.transform, "Label", "+", 22, Color.white);
    Stretch(plusL.rectTransform);

    backpackGrid = gameObject.GetComponent<TownBackpackGrid>();
    if (backpackGrid == null) backpackGrid = gameObject.AddComponent<TownBackpackGrid>();
    backpackGrid.BuildGrid(bag.transform);

    // 底部五入口占位（不自建按钮，仅留空给 SharedChrome）
    var reserve = CreateRect(transform, "BottomNavReserve");
    var rrt = reserve.GetComponent<RectTransform>();
    rrt.anchorMin = new Vector2(0f, 0f);
    rrt.anchorMax = new Vector2(1f, 0f);
    rrt.pivot = new Vector2(0.5f, 0f);
    rrt.sizeDelta = new Vector2(0f, BottomReserve);
    rrt.anchoredPosition = Vector2.zero;

    // Skill select popup
    var skillGo = new GameObject("SkillSelectUI", typeof(RectTransform));
    skillGo.transform.SetParent(transform, false);
    Stretch(skillGo.GetComponent<RectTransform>());
    skillSelect = skillGo.AddComponent<SkillSelectUI>();
    skillSelect.BuildHierarchyForPrefab();

    AutoBind();
    GameFonts.ApplyToHierarchy(transform);
    _built = true;
}

static void BuildSideBtn(Transform parent, string name, string label, float y, Color col)
{
    var img = CreateImg(parent, name, col);
    Set(img.rectTransform, 1f, 1f, 1f, 1f, 1f, 1f, 0f, y, 110f, 72f);
    img.gameObject.AddComponent<Button>().targetGraphic = img;
    var t = CreateTxt(img.transform, "Label", label, 24, Color.white);
    Stretch(t.rectTransform);
}

static void CreateAttr(Transform parent, string name, string label, string value, float index)
{
    float x0 = index / 6f;

```

----- 第 38 页 -----

```

float x1 = (index + 1f) / 6f;
var root = CreateRect(parent, name + "Root");
var rt = root.GetComponent<RectTransform>();
rt.anchorMin = new Vector2(x0, 0f);
rt.anchorMax = new Vector2(x1, 0.7f);
rt.offsetMin = new Vector2(4f, 4f);
rt.offsetMax = new Vector2(-4f, -4f);
var lab = CreateTxt(root.transform, "Label", label, 16, new Color(0.85f, 0.8f, 0.7f));
Set(lab.rectTransform, 0f, 0.55f, 1f, 1f, 0.5f, 0.5f, 0f, 0f, 0f, 0f);
// 数值节点名与 AutoBind 一致
var val = CreateTxt(root.transform, name, value, 20, new Color(0.7f, 0.95f, 0.7f));
Set(val.rectTransform, 0f, 0f, 1f, 0.55f, 0.5f, 0.5f, 0f, 0f, 0f, 0f);
}

static GameObject CreateRect(Transform p, string n)
{
    var go = new GameObject(n, typeof(RectTransform));
    go.transform.SetParent(p, false);
    return go;
}

static Image CreateImg(Transform p, string n, Color c)
{
    var go = new GameObject(n, typeof(RectTransform), typeof(CanvasRenderer), typeof(Image));
    go.transform.SetParent(p, false);
    var img = go.GetComponent<Image>();
    img.color = c;
    return img;
}

static Text CreateTxt(Transform p, string n, string t, int size, Color c)
{
    var go = new GameObject(n, typeof(RectTransform), typeof(CanvasRenderer), typeof(Text));
    go.transform.SetParent(p, false);
    var tx = go.GetComponent<Text>();
    tx.text = t;
    tx.fontSize = size;
    tx.color = c;
    tx.alignment = TextAnchor.MiddleCenter;
    tx.raycastTarget = false;
    tx.font = GameFonts.GetChinese();
    return tx;
}

static void Stretch(RectTransform rt)
{
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = Vector2.zero;
    rt.offsetMax = Vector2.zero;
}

```

----- 第 39 页 -----

```

    }

    static void Set(RectTransform rt, float aminX, float aminY, float amaxX, float amaxY,
        float px, float py, float x, float y, float w, float h)
    {
        rt.anchorMin = new Vector2(aminX, aminY);
        rt.anchorMax = new Vector2(amaxX, amaxY);
        rt.pivot = new Vector2(px, py);
        rt.anchoredPosition = new Vector2(x, y);
        rt.sizeDelta = new Vector2(w, h);
    }
}

```

```

// ===== FILE: Assets/Scripts/UI/BattleStageMapUI.cs =====
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// 战斗内关卡石墩图。
///
/// 流程：进图 → 下一关石墩上的锁晃两下再放大消失 → 滚盘选类型
/// → 类型旗落到当前石墩并挂 btn06 描边 → 玩家点石墩进入。
///
/// 石墩三态：
/// • 当前可打：Banner + Icon + btn06 描边，可点
/// • 已打过：ClearedMark（变灰旗子），不可点
/// • 未解锁：只有石墩 + Lock，无旗子
///
/// 从下往上打：Pedestal_0 在最下面，默认无锁。
/// </summary>
public class BattleStageMapUI : MonoBehaviour
{
    public const string PrefabPath = "Prefabs/Battle/BattleStageMap";
    public const int PedestalCount = 10;

    public static BattleStageMapUI Instance { get; private set; }

    [Header("可绑节点")]
    public GameObject root;
    public Image backdrop;
    public Text titleText;
    public Transform pedestalRoot;
    public List<PedestalRefs> pedestals = new List<PedestalRefs>();
    [Tooltip("当前关描边：拖 btn06。空则试 Resources/Materials/btn06")]
    public Material currentOutlineMaterial;

    [Serializable]

```

----- 第 40 页 -----

```

public class PedestalRefs
{
    public GameObject root;
    public Button button;
    public Image stone;
    public Image banner;
    public Image icon;
    /// <summary>已通关的灰旗（ClearedMark 节点本身就是灰旗图）</summary>
    public GameObject clearedMark;
    /// <summary>未解锁锁头：解锁动画播在这个节点上</summary>
    public GameObject lockIcon;
    public Image outlineTarget; // 挂 btn06 的 Image，默认用 Banner 或 Stone
}

StageData _next;
Action<StageData> _onPick;
float _prevTimeScale = 1f;
Coroutine _flow;
Coroutine _backdropPulse;
Color _backdropBase = Color.white;
Material _btn06;
readonly Dictionary<Image, Material> _savedMats = new Dictionary<Image, Material>();

/// <summary>进关卡图并跑完整流程（解锁 → 滚盘 → 落旗 → 等点击）。</summary>
public static void BeginFlow(StageData next, Action<StageData> onPick)
{
    if (next == null)
    {
        onPick?.Invoke(null);
        return;
    }
    Ensure().StartFlow(next, onPick);
}

public static BattleStageMapUI Ensure()
{
    if (Instance != null) return Instance;

    var prefab = Resources.Load<GameObject>(PrefabPath);
    GameObject go;
    if (prefab != null)
    {
        go = Instantiate(prefab);
        go.name = "BattleStageMap";
    }
    else
    {
        Debug.LogWarning($"[BattleStageMap] 未找到预制体 {PrefabPath}，代码搭建临时界面");
        go = new GameObject("BattleStageMap", typeof(RectTransform));
        BuildHierarchy(go);
    }
}

```


----- 第 41 页 -----

```
    }

    DontDestroyOnLoad(go);

    var ui = go.GetComponent<BattleStageMapUI>();
    if (ui == null) ui = go.AddComponent<BattleStageMapUI>();
    return ui;
}

void Awake()
{
    Instance = this;
    BindRefs();
    if (root != null) root.SetActive(false);
}

void OnDestroy()
{
    if (Instance == this) Instance = null;
}

void StartFlow(StageData next, Action<StageData> onPick)
{
    BindRefs();
    _next = next;
    _onPick = onPick;

    EnsureCanvas();
    _prevTimeScale = Time.timeScale;
    Time.timeScale = 0f;

    int chapter = ChapterManager.Instance != null ? ChapterManager.Instance.currentChapter : 1;
    if (titleText != null)
        titleText.text = GameConfig.GetChapterTitleText(chapter);

    // 滚盘前: 当前关只显示锁 (第 0 关除外), 已通关灰旗, 未解锁光墩
    RefreshPedestals(Phase.BeforeRoll);

    if (root != null) root.SetActive(true);
    transform.SetAsLastSibling();
    GameFonts.ApplyToHierarchy(transform);

    StartBackdropPulse();
    if (_flow != null) StopCoroutine(_flow);
    _flow = StartCoroutine(CoFlow());
}

enum Phase
{
    BeforeRoll, // 下一关: 锁; 已通关: 灰旗; 未来: 光墩
    AfterRoll   // 下一关: 彩色旗 + btn06; 其余同上
}
```

----- 第 42 页 -----

```

    }

IEnumerator CoFlow()
{
    int nextIdx = _next != null ? _next.stageIndex : 0;

    // 第 0 关默认没锁，直接进滚盘；其它关先播解锁
    if (nextIdx > 0)
        yield return PlayUnlockAnim(nextIdx);
    else
        yield return WaitUnscaled(0.25f);

    // 滚盘（类型已抽好，这里只演出）
    int chapter = ChapterManager.Instance != null ? ChapterManager.Instance.currentChapter : 1;
    bool rolled = false;
    StageData picked = _next;
    NextStageRouletteUI.Show(chapter, _next, p =>
    {
        if (p != null) picked = p;
        rolled = true;
    });
    while (!rolled) yield return null;

    _next = picked;
    // 类型旗落到当前石墩 + btn06 描边
    RefreshPedestals(Phase.AfterRoll);
    ApplyCurrentOutline(nextIdx);

    _flow = null;
    // 之后等玩家点石墩（OnPedestalClicked）
}

IEnumerator PlayUnlockAnim(int index)
{
    if (index < 0 || index >= pedestals.Count) yield break;
    var p = pedestals[index];
    if (p == null || p.lockIcon == null)
    {
        yield return WaitUnscaled(0.2f);
        yield break;
    }

    var lockTf = p.lockIcon.transform;
    p.lockIcon.SetActive(true);
    Vector3 baseScale = Vector3.one;
    Vector3 basePos = lockTf.localPosition;
    lockTf.localScale = baseScale;

    // 晃两下
    for (int shake = 0; shake < 2; shake++)

```

----- 第 43 页 -----

```

{
    float t = 0f;
    const float dur = 0.22f;
    while (t < 1f)
    {
        t += Time.unscaledDeltaTime / dur;
        float w = Mathf.Sin(t * Mathf.PI * 4f) * (1f - t) * 12f;
        lockTf.localPosition = basePos + new Vector3(w, 0f, 0f);
        lockTf.localRotation = Quaternion.Euler(0f, 0f, w * 0.8f);
        yield return null;
    }
    lockTf.localPosition = basePos;
    lockTf.localRotation = Quaternion.identity;
    yield return WaitUnscaled(0.06f);
}

// 放大消失
float u = 0f;
const float pop = 0.35f;
while (u < 1f)
{
    u += Time.unscaledDeltaTime / pop;
    float e = Mathf.Clamp01(u);
    lockTf.localScale = baseScale * (1f + e * 1.4f);
    SetLockAlpha(p.lockIcon, 1f - e);
    yield return null;
}

p.lockIcon.SetActive(false);
lockTf.localScale = baseScale;
lockTf.localPosition = basePos;
SetLockAlpha(p.lockIcon, 1f);
}

static void SetLockAlpha(GameObject lockGo, float a)
{
    if (lockGo == null) return;
    var imgs = lockGo.GetComponentsInChildren<Image>(true);
    for (int i = 0; i < imgs.Length; i++)
    {
        var c = imgs[i].color;
        c.a = a;
        imgs[i].color = c;
    }
}

void RefreshPedestals(Phase phase)
{
    var cm = ChapterManager.Instance;
    int currentDone = cm != null ? cm.currentStageIndex : -1;

```

----- 第 44 页 -----

```
// 刚通关后 currentStageIndex 还是刚打完的那关
int nextIdx = _next != null ? _next.stageIndex : currentDone + 1;

ClearAllOutlines();

for (int i = 0; i < pedestals.Count; i++)
{
    var p = pedestals[i];
    if (p == null || p.root == null) continue;

    bool cleared = i <= currentDone;
    bool isNext = i == nextIdx;
    bool locked = i > nextIdx;

    // 永远关掉旧的 Highlight (如果预制体里还留着)
    var hi = p.root.transform.Find("Highlight");
    if (hi != null) hi.gameObject.SetActive(false);

    if (isNext)
    {
        if (phase == Phase.BeforeRoll)
        {
            // 滚盘前: 第 0 关无锁直接可点外观; 其它关显示锁、暂无旗
            bool showLock = nextIdx > 0;
            SetFlag(p, false, false);
            if (p.clearedMark != null) p.clearedMark.SetActive(false);
            if (p.lockIcon != null) p.lockIcon.SetActive(showLock);
            if (p.stone != null) p.stone.color = Color.white;
            // 解锁动画播完前先不可点; 第 0 关可先不可点等滚盘
            SetClickable(p, false, i);
        }
        else
        {
            // 滚盘后: 彩色旗 + 可点
            if (p.lockIcon != null) p.lockIcon.SetActive(false);
            if (p.clearedMark != null) p.clearedMark.SetActive(false);
            SetFlag(p, true, false);
            if (_next != null) ApplyTypeIcon(p, _next.type);
            if (p.stone != null) p.stone.color = Color.white;
            SetClickable(p, true, i);
        }
    }
    else if (cleared)
    {
        // 已通关: 灰旗 (ClearedMark)
        if (p.lockIcon != null) p.lockIcon.SetActive(false);
        SetFlag(p, false, false);
        if (p.clearedMark != null) p.clearedMark.SetActive(true);
        if (p.stone != null) p.stone.color = new Color(0.85f, 0.85f, 0.85f, 1f);
        SetClickable(p, false, i);
    }
}
```

----- 第 45 页 -----

```
    }
    else
    {
        // 未解锁：只有石墩 + 锁，无旗
        SetFlag(p, false, false);
        if (p.clearedMark != null) p.clearedMark.SetActive(false);
        if (p.lockIcon != null) p.lockIcon.SetActive(true);
        if (p.stone != null) p.stone.color = new Color(0.55f, 0.55f, 0.55f, 0.95f);
        SetClickable(p, false, i);
    }
}

}

void SetFlag(PedestalRefs p, bool showBanner, bool gray)
{
    if (p.banner != null)
    {
        p.banner.gameObject.SetActive(showBanner);
        if (showBanner)
            p.banner.color = gray ? new Color(0.45f, 0.45f, 0.45f, 1f) : Color.white;
    }
    if (p.icon != null)
        p.icon.gameObject.SetActive(showBanner);
}

void SetClickable(PedestalRefs p, bool on, int index)
{
    if (p.button == null) return;
    p.button.onClick.RemoveAllListeners();
    p.button.interactable = on;
    if (on)
    {
        int capture = index;
        p.button.onClick.AddListener(() => OnPedestalClicked(capture));
    }
}

void ApplyTypeIcon(PedestalRefs p, StageType type)
{
    Sprite sp = LoadStageIcon(type);
    if (p.icon != null)
    {
        if (sp != null)
        {
            p.icon.sprite = sp;
            p.icon.color = Color.white;
            p.icon.preserveAspect = true;
        }
        else
            p.icon.color = StageRoller.Tint(type);
    }
}
```

----- 第 46 页 -----

```
    }
    if (p.banner != null)
        p.banner.color = Color.white;
}

void ApplyCurrentOutline(int index)
{
    if (index < 0 || index >= pedestals.Count) return;
    var p = pedestals[index];
    if (p == null) return;

    Image target = p.outlineTarget;
    if (target == null) target = p.banner != null ? p.banner : p.stone;
    if (target == null) return;

    Material mat = ResolveBtn06();
    if (mat == null)
    {
        Debug.LogWarning("[BattleStageMap] 找不到 btn06 材质, 请在组件上拖入 currentOutlineMaterial");
        return;
    }
    if (!_savedMats.ContainsKey(target))
        _savedMats[target] = target.material;
    target.material = mat;
}

void ClearAllOutlines()
{
    foreach (var kv in _savedMats)
    {
        if (kv.Key != null)
            kv.Key.material = kv.Value;
    }
    _savedMats.Clear();
}

Material ResolveBtn06()
{
    if (currentOutlineMaterial != null) return currentOutlineMaterial;
    if (_btn06 != null) return _btn06;
    _btn06 = Resources.Load<Material>("Materials/btn06");
    if (_btn06 == null) _btn06 = Resources.Load<Material>("btn06");
    return _btn06;
}

void OnPedestalClicked(int index)
{
    if (_next == null || index != _next.stageIndex) return;
    if (_flow != null) return; // 还在解锁/滚盘中
```

----- 第 47 页 -----

```

        StopBackdropPulse();
        ClearAllOutlines();
        Time.timeScale = _prevTimeScale > 0.01f ? _prevTimeScale : 1f;
        if (root != null) root.SetActive(false);

        var stage = _next;
        var cb = _onPick;
        _onPick = null;
        _next = null;
        cb?.Invoke(stage);
    }

    public void Hide()
    {
        if (_flow != null)
        {
            StopCoroutine(_flow);
            _flow = null;
        }
        StopBackdropPulse();
        ClearAllOutlines();
        if (root != null) root.SetActive(false);
        Time.timeScale = _prevTimeScale > 0.01f ? _prevTimeScale : 1f;
    }

    // ===== Backdrop 灯光昏暗抖动 =====

    void StartBackdropPulse()
    {
        if (backdrop == null) return;
        _backdropBase = backdrop.color;
        if (_backdropPulse != null) StopCoroutine(_backdropPulse);
        _backdropPulse = StartCoroutine(CoBackdropPulse());
    }

    void StopBackdropPulse()
    {
        if (_backdropPulse != null)
        {
            StopCoroutine(_backdropPulse);
            _backdropPulse = null;
        }
        if (backdrop != null) backdrop.color = _backdropBase;
    }

    IEnumerator CoBackdropPulse()
    {
        // 颜色略变深再回来，不规则一点像火把抖动
        while (true)
        {

```

----- 第 48 页 -----

```

float deep = UnityEngine.Random.Range(0.72f, 0.88f);
float toDark = UnityEngine.Random.Range(0.35f, 0.7f);
float toLight = UnityEngine.Random.Range(0.4f, 0.9f);
yield return LerpBackdrop(_backdropBase, _backdropBase * deep, toDark);
yield return LerpBackdrop(_backdropBase * deep, _backdropBase, toLight);
// 偶尔多顿一下
if (UnityEngine.Random.value < 0.35f)
{
    float flick = UnityEngine.Random.Range(0.65f, 0.8f);
    yield return LerpBackdrop(_backdropBase, _backdropBase * flick, 0.12f);
    yield return LerpBackdrop(_backdropBase * flick, _backdropBase, 0.18f);
}
}
}

```

```
IEnumerator LerpBackdrop(Color from, Color to, float seconds)
```

```

{
    float t = 0f;
    seconds = Mathf.Max(0.05f, seconds);
    while (t < 1f)
    {
        t += Time.unscaledDeltaTime / seconds;
        if (backdrop != null)
        {
            Color c = Color.Lerp(from, to, Mathf.Clamp01(t));
            c.a = _backdropBase.a;
            backdrop.color = c;
        }
        yield return null;
    }
}

```

```
static IEnumerator WaitUnscaled(float seconds)
```

```

{
    float t = 0f;
    while (t < seconds)
    {
        t += Time.unscaledDeltaTime;
        yield return null;
    }
}

```

```
// ===== 绑定 =====
```

```
void BindRefs()
```

```

{
    if (root == null)
    {
        var t = transform.Find("Root");
        root = t != null ? t.gameObject : gameObject;
    }
}

```


----- 第 49 页 -----

```

    }
    if (backdrop == null)
    {
        var t = root.transform.Find("Backdrop");
        if (t != null) backdrop = t.GetComponent<Image>();
    }
    if (titleText == null)
    {
        var t = FindDeep(root.transform, "Title");
        if (t != null) titleText = t.GetComponent<Text>();
    }
    if (pedestalRoot == null)
    {
        var t = root.transform.Find("Pedestals");
        if (t != null) pedestalRoot = t;
    }

    if (pedestals == null) pedestals = new List<PedestalRefs>();
    if (pedestals.Count < PedestalCount && pedestalRoot != null)
    {
        pedestals.Clear();
        for (int i = 0; i < PedestalCount; i++)
        {
            Transform p = pedestalRoot.Find($"Pedestal_{i}")
                ?? pedestalRoot.Find($"Stage_{i}")
                ?? pedestalRoot.Find($"Node_{i}");
            if (p == null) continue;
            pedestals.Add(BindOne(p.gameObject));
        }
    }
    else
    {
        for (int i = 0; i < pedestals.Count; i++)
        {
            if (pedestals[i] != null && pedestals[i].root != null
                && pedestals[i].lockIcon == null)
            {
                // 补绑 Lock
                var L = pedestals[i].root.transform.Find("Lock")
                    ?? pedestals[i].root.transform.Find("LockIcon");
                if (L != null) pedestals[i].lockIcon = L.gameObject;
            }
        }
    }
}

static PedestalRefs BindOne(GameObject go)
{
    var r = new PedestalRefs { root = go };
    r.button = go.GetComponent<Button>() ?? go.GetComponentInChildren<Button>(true);
}

```

----- 第 50 页 -----

```

    r.stone = go.transform.Find("Stone").GetComponent<Image>()
    ?? go.GetComponent<Image>();
    r.banner = go.transform.Find("Banner").GetComponent<Image>();
    r.icon = go.transform.Find("Icon").GetComponent<Image>()
    ?? go.transform.Find("Banner/Icon").GetComponent<Image>();
    var cleared = go.transform.Find("ClearedMark");
    if (cleared != null) r.clearedMark = cleared.gameObject;
    var lockT = go.transform.Find("Lock") ?? go.transform.Find("LockIcon");
    if (lockT != null) r.lockIcon = lockT.gameObject;
    r.outlineTarget = r.banner != null ? r.banner : r.stone;
    // Highlight 不再使用
    var hi = go.transform.Find("Highlight");
    if (hi != null) hi.gameObject.SetActive(false);
    return r;
}

```

```

void EnsureCanvas()
{
    var canvas = GetComponent<Canvas>();
    if (canvas == null) canvas = gameObject.AddComponent<Canvas>();
    UICanvasSetup.Apply(canvas);
    canvas.sortingOrder = 880;
    if (GetComponent<GraphicRaycaster>() == null)
        gameObject.AddComponent<GraphicRaycaster>();
    EnsureEventSystem();
}

```

```

static void EnsureEventSystem()
{
    if (UnityEngine.EventSystems.EventSystem.current != null) return;
    var go = new GameObject("EventSystem",
        typeof(UnityEngine.EventSystems.EventSystem),
        typeof(UnityEngine.EventSystems.StandaloneInputModule));
    go.hideFlags = HideFlags.DontSave;
}

```

```

static Sprite LoadStageIcon(StageType type)
{
    string n = StageRoller.IconName(type);
    var sp = Resources.Load<Sprite>($"Art/UI/StageIcons/{n}");
    if (sp == null) sp = Resources.Load<Sprite>($"UI/StageIcons/{n}");
    if (sp == null) sp = Resources.Load<Sprite>($"Art/UI/关卡/{n}");
    return sp;
}

```

// ===== 预制体结构（从下往上：0 在最底）=====

```

public static void BuildHierarchy(GameObject host)
{
    if (host == null) return;

```

----- 第 51 页 -----

```

    if (host.GetComponent<RectTransform>() == null)
        host.AddComponent<RectTransform>();

    for (int i = host.transform.childCount - 1; i >= 0; i--)
        UnityEngine.Object.DestroyImmediate(host.transform.GetChild(i).gameObject);

    var rootGo = new GameObject("Root", typeof(RectTransform));
    rootGo.transform.SetParent(host.transform, false);
    Stretch(rootGo.GetComponent<RectTransform>());

    var bg = NewImage(rootGo.transform, "Backdrop", new Color(0.12f, 0.08f, 0.16f, 1f));
    Stretch(bg.rectTransform);
    var mapSp = Resources.Load<Sprite>("Art/UI/StageSelect/bg_stage_select");
    if (mapSp != null)
    {
        bg.sprite = mapSp;
        bg.color = Color.white;
    }

    var title = NewText(rootGo.transform, "Title", "第X章", 36, TextAnchor.MiddleCenter);
    title.color = new Color(0.85f, 0.75f, 1f, 1f);
    var trt = title.rectTransform;
    trt.anchorMin = trt.anchorMax = new Vector2(0.5f, 1f);
    trt.pivot = new Vector2(0.5f, 1f);
    trt.anchoredPosition = new Vector2(0f, -80f);
    trt.sizeDelta = new Vector2(560f, 50f);

    var tip = NewText(rootGo.transform, "Tip", "点击石墩上的关卡继续冒险", 22, TextAnchor.MiddleCenter);
    tip.color = new Color(0.8f, 0.75f, 0.65f, 1f);
    var tipRt = tip.rectTransform;
    tipRt.anchorMin = tipRt.anchorMax = new Vector2(0.5f, 1f);
    tipRt.pivot = new Vector2(0.5f, 1f);
    tipRt.anchoredPosition = new Vector2(0f, -130f);
    tipRt.sizeDelta = new Vector2(560f, 36f);

    var pedRoot = new GameObject("Pedestals", typeof(RectTransform));
    pedRoot.transform.SetParent(rootGo.transform, false);
    Stretch(pedRoot.GetComponent<RectTransform>());

    // Pedestal_0 在最下面, 往上到 Pedestal_9
    Vector2[] pos =
    {
        new Vector2(20f, -440f), // 0 最底
        new Vector2(-80f, -350f),
        new Vector2(60f, -260f),
        new Vector2(-40f, -170f),
        new Vector2(80f, -80f),
        new Vector2(-70f, 10f),
        new Vector2(50f, 100f),
        new Vector2(-90f, 190f),
    }

```

----- 第 52 页 -----

```
new Vector2( 70f, 280f),
new Vector2(-30f, 380f), // 9 最上
};
```

```
var ui = host.GetComponent<BattleStageMapUI>() ?? host.AddComponent<BattleStageMapUI>();
ui.root = rootGo;
ui.backdrop = bg;
ui.titleText = title;
ui.pedestalRoot = pedRoot.transform;
ui.pedestals = new List<PedestalRefs>();
ui.currentOutlineMaterial = Resources.Load<Material>("Materials/btn06");
```

```
for (int i = 0; i < PedestalCount; i++)
    ui.pedestals.Add(BuildPedestal(pedRoot.transform, i, pos[i]));
```

```
GameFonts.ApplyToHierarchy(host.transform);
```

```
}
```

```
static PedestalRefs BuildPedestal(Transform parent, int index, Vector2 anchored)
```

```
{
```

```
var go = new GameObject($"Pedestal_{index}", typeof(RectTransform), typeof(Image), typeof(Button));
go.transform.SetParent(parent, false);
var rt = go.GetComponent<RectTransform>();
rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
rt.pivot = new Vector2(0.5f, 0f);
rt.sizeDelta = new Vector2(120f, 100f);
rt.anchoredPosition = anchored;
```

```
var stone = go.GetComponent<Image>();
stone.color = new Color(0.45f, 0.4f, 0.38f, 1f);
var btn = go.GetComponent<Button>();
btn.targetGraphic = stone;
```

```
var banner = NewImage(rt, "Banner", Color.white);
var brt = banner.rectTransform;
brt.anchorMin = brt.anchorMax = new Vector2(0.5f, 1f);
brt.pivot = new Vector2(0.5f, 0f);
brt.sizeDelta = new Vector2(72f, 88f);
brt.anchoredPosition = new Vector2(0f, 8f);
banner.gameObject.SetActive(false);
```

```
var icon = NewImage(brt, "Icon", Color.white);
var irt = icon.rectTransform;
irt.anchorMin = irt.anchorMax = new Vector2(0.5f, 0.5f);
irt.sizeDelta = new Vector2(48f, 48f);
irt.anchoredPosition = new Vector2(0f, 6f);
icon.preserveAspect = true;
```

```
// ClearedMark = 灰旗 (换灰旗图)
```

```
var cleared = NewImage(rt, "ClearedMark", new Color(0.4f, 0.4f, 0.4f, 1f));
```

----- 第 53 页 -----

```
var crt = cleared.rectTransform;
crt.anchorMin = crt.anchorMax = new Vector2(0.5f, 1f);
crt.pivot = new Vector2(0.5f, 0f);
crt.sizeDelta = new Vector2(72f, 88f);
crt.anchoredPosition = new Vector2(0f, 8f);
cleared.gameObject.SetActive(false);

// Lock
var lockImg = NewImage(rt, "Lock", new Color(0.9f, 0.75f, 0.2f, 1f));
var lrt = lockImg.rectTransform;
lrt.anchorMin = lrt.anchorMax = new Vector2(0.5f, 0.5f);
lrt.sizeDelta = new Vector2(48f, 48f);
lrt.anchoredPosition = new Vector2(0f, 28f);
lockImg.gameObject.SetActive(false);

// 不生成 Highlight

return new PedestalRefs
{
    root = go,
    button = btn,
    stone = stone,
    banner = banner,
    icon = icon,
    clearedMark = cleared.gameObject,
    lockIcon = lockImg.gameObject,
    outlineTarget = banner
};
}

static Transform FindDeep(Transform root, string name)
{
    if (root == null) return null;
    if (root.name == name) return root;
    for (int i = 0; i < root.childCount; i++)
    {
        var f = FindDeep(root.GetChild(i), name);
        if (f != null) return f;
    }
    return null;
}

static void Stretch(RectTransform rt)
{
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = Vector2.zero;
    rt.offsetMax = Vector2.zero;
}
```

----- 第 54 页 -----

```

static Image NewImage(Transform parent, string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(CanvasRenderer), typeof(Image));
    go.transform.SetParent(parent, false);
    var img = go.GetComponent<Image>();
    img.color = color;
    img.raycastTarget = false;
    return img;
}

static Text NewText(Transform parent, string name, string content, int size, TextAnchor align)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(CanvasRenderer), typeof(Text));
    go.transform.SetParent(parent, false);
    var t = go.GetComponent<Text>();
    t.font = GameFonts.GetChinese();
    t.text = content;
    t.fontSize = size;
    t.alignment = align;
    t.color = Color.white;
    t.raycastTarget = false;
    t.horizontalOverflow = HorizontalWrapMode.Overflow;
    t.verticalOverflow = VerticalWrapMode.Overflow;
    return t;
}
}

// ===== FILE: Assets/Scripts/UI/OfflineRewardPopup.cs =====
using System;
using UnityEngine;
using UnityEngine.UI;

/// <summary>
/// 离线收益弹窗（农场金币）。
/// </summary>
public class OfflineRewardPopup : MonoBehaviour
{
    public static OfflineRewardPopup Instance { get; private set; }

    GameObject _root;
    Text _body;
    Action _onClose;

    public static void Show(long gold, double minutes, Action onClose = null)
    {
        if (gold <= 0)
        {
            onClose?.Invoke();
            return;
        }
    }
}

```

----- 第 55 页 -----

```

        Ensure().Open(gold, minutes, onClose);
    }

    static OfflineRewardPopup Ensure()
    {
        if (Instance != null) return Instance;
        var go = new GameObject("OfflineRewardPopup");
        DontDestroyOnLoad(go);
        return go.AddComponent<OfflineRewardPopup>();
    }

    void Awake() => Instance = this;

    void OnDestroy()
    {
        if (Instance == this) Instance = null;
    }

    void Open(long gold, double minutes, Action onClose)
    {
        _onClose = onClose;
        BuildIfNeeded();
        if (_body != null)
            _body.text = $"离线 {minutes:F0} 分钟\n已到账金币 +{gold}";
        if (_root != null)
        {
            _root.SetActive(true);
            _root.transform.SetAsLastSibling();
        }
        GameFonts.ApplyToHierarchy(transform);
    }

    void BuildIfNeeded()
    {
        if (_root != null) return;
        var canvas = gameObject.GetComponent<Canvas>() ?? gameObject.AddComponent<Canvas>();
        if (gameObject.GetComponent<GraphicRaycaster>() == null)
            gameObject.AddComponent<GraphicRaycaster>();
        UICanvasSetup.Apply(canvas);
        canvas.sortingOrder = 980;

        _root = new GameObject("Root", typeof(RectTransform));
        _root.transform.SetParent(transform, false);
        var rootRt = _root.GetComponent<RectTransform>();
        rootRt.anchorMin = Vector2.zero;
        rootRt.anchorMax = Vector2.one;
        rootRt.offsetMin = rootRt.offsetMax = Vector2.zero;

        var dim = new GameObject("Dim", typeof(RectTransform), typeof(Image));
        dim.transform.SetParent(_root.transform, false);
    }

```

----- 第 56 页 -----

```

var drt = dim.GetComponent<RectTransform>();
drt.anchorMin = Vector2.zero;
drt.anchorMax = Vector2.one;
drt.offsetMin = drt.offsetMax = Vector2.zero;
dim.GetComponent<Image>().color = new Color(0f, 0f, 0f, 0.55f);

var panel = new GameObject("Panel", typeof(RectTransform), typeof(Image));
panel.transform.SetParent(_root.transform, false);
var prt = panel.GetComponent<RectTransform>();
prt.anchorMin = prt.anchorMax = new Vector2(0.5f, 0.5f);
prt.sizeDelta = new Vector2(420f, 280f);
panel.GetComponent<Image>().color = new Color(0.12f, 0.1f, 0.14f, 0.96f);

var titleGo = new GameObject("Title", typeof(RectTransform), typeof(Text));
titleGo.transform.SetParent(panel.transform, false);
var trt = titleGo.GetComponent<RectTransform>();
trt.anchorMin = new Vector2(0f, 1f);
trt.anchorMax = new Vector2(1f, 1f);
trt.pivot = new Vector2(0.5f, 1f);
trt.anchoredPosition = new Vector2(0f, -20f);
trt.sizeDelta = new Vector2(-40f, 40f);
var title = titleGo.GetComponent<Text>();
title.text = "离线收益";
title.fontSize = 30;
title.alignment = TextAnchor.MiddleCenter;
title.color = Color.white;
if (title.font == null) title.font = Resources.GetBuiltinResource<Font>("Arial.ttf");

var bodyGo = new GameObject("Body", typeof(RectTransform), typeof(Text));
bodyGo.transform.SetParent(panel.transform, false);
var brt = bodyGo.GetComponent<RectTransform>();
brt.anchorMin = new Vector2(0.1f, 0.35f);
brt.anchorMax = new Vector2(0.9f, 0.75f);
brt.offsetMin = brt.offsetMax = Vector2.zero;
_body = bodyGo.GetComponent<Text>();
_body.fontSize = 24;
_body.alignment = TextAnchor.MiddleCenter;
_body.color = Color.white;
if (_body.font == null) _body.font = Resources.GetBuiltinResource<Font>("Arial.ttf");

var btn = new GameObject("Ok", typeof(RectTransform), typeof(Image), typeof(Button));
btn.transform.SetParent(panel.transform, false);
var btnRt = btn.GetComponent<RectTransform>();
btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0f);
btnRt.pivot = new Vector2(0.5f, 0f);
btnRt.anchoredPosition = new Vector2(0f, 28f);
btnRt.sizeDelta = new Vector2(180f, 48f);
btn.GetComponent<Image>().color = new Color(0.25f, 0.55f, 0.35f, 1f);
var btnTxtGo = new GameObject("T", typeof(RectTransform), typeof(Text));
btnTxtGo.transform.SetParent(btn.transform, false);

```


----- 第 57 页 -----

```

var btnTxtRt = btnTxtGo.GetComponent<RectTransform>();
btnTxtRt.anchorMin = Vector2.zero;
btnTxtRt.anchorMax = Vector2.one;
btnTxtRt.offsetMin = btnTxtRt.offsetMax = Vector2.zero;
var btnTxt = btnTxtGo.GetComponent<Text>();
btnTxt.text = "确定";
btnTxt.fontSize = 24;
btnTxt.alignment = TextAnchor.MiddleCenter;
btnTxt.color = Color.white;
if (btnTxt.font == null) btnTxt.font = Resources.GetBuiltinResource<Font>("Arial.ttf");
btn.GetComponent<Button>().onClick.AddListener(Close);
}

void Close()
{
    if (_root != null) _root.SetActive(false);
    var cb = _onClose;
    _onClose = null;
    cb?.Invoke();
}
}

```

// ===== FILE: Assets/Scripts/Combat/DamageFormula.cs =====

using UnityEngine;

```

/// <summary>
/// 统一伤害结算：ATK/技能伤害 → 暴击 → 减伤/防御 → 下限 1。
/// 普攻与 SkillSystem 都走这里，避免多处公式漂移。
/// </summary>
public static class DamageFormula
{
    public const float MinDamage = 1f;

    /// <summary>暴击倍率 = 1.5 + BASE_CRIT_DAMAGE（与数值表一致）。</summary>
    public static float CritMultiplier(float critDamageBonus = 0f)
    {
        return 1.5f + GameConfig.BASE_CRIT_DAMAGE + Mathf.Max(0f, critDamageBonus);
    }

    /// <summary>是否暴击。</summary>
    public static bool RollCrit(AttrSystem attacker)
    {
        if (attacker == null) return false;
        return Random.value < attacker.GetAttr(AttrType.CritRate);
    }

    /// <summary>
    /// 从攻击者攻击力生成「击中前」伤害（已含暴击；尚未扣防）。
    /// </summary>
    public static float BuildAttackRaw(AttrSystem attacker, out bool isCrit)

```

----- 第 58 页 -----

```

{
    isCrit = false;
    if (attacker == null) return MinDamage;
    float damage = attacker.GetAttr(AttrType.Attack);
    isCrit = RollCrit(attacker);
    if (isCrit)
        damage *= CritMultiplier();
    return Mathf.Max(MinDamage, damage);
}

/// <summary>
/// 技能基础伤害（未暴击）：base + ATK * mul * (1+物魔强)。
/// </summary>
public static float BuildSkillBase(float baseDamage, float atkMul, AttrSystem attacker)
{
    if (attacker == null) return Mathf.Max(MinDamage, baseDamage);
    float attack = attacker.GetAttr(AttrType.Attack);
    float phy = attacker.GetAttr(AttrType.PhyPower);
    float mag = attacker.GetAttr(AttrType.MagicPower);
    return Mathf.Max(MinDamage, baseDamage + attack * atkMul * (1f + phy + mag));
}

/// <summary>对技能基础伤害应用暴击。</summary>
public static float ApplyCrit(float baseDamage, AttrSystem attacker, out bool isCrit)
{
    isCrit = RollCrit(attacker);
    if (!isCrit) return Mathf.Max(MinDamage, baseDamage);
    return Mathf.Max(MinDamage, baseDamage * CritMultiplier());
}

/// <summary>
/// 最终扣血量：raw 已含暴击；再减 DEF。
/// ignoreDefense：引导等特殊命中。
/// </summary>
public static float FinalHit(float rawDamage, AttrSystem defender, bool ignoreDefense = false)
{
    float dmg = Mathf.Max(0f, rawDamage);
    if (!ignoreDefense && defender != null)
    {
        float def = defender.GetAttr(AttrType.Defense);
        dmg = Mathf.Max(MinDamage, dmg - def);
    }
    return Mathf.Max(MinDamage, dmg);
}
}

// ===== FILE: Assets/Scripts/Platform/CloudSaveBridge.cs =====
using System;
using System.IO;
using UnityEngine;

```

----- 第 59 页 -----

```
/// <summary>
/// 云存档适配层：本地文件镜像 + 微信云预留接口。
/// </summary>
public static class CloudSaveBridge
{
    const string CloudMirrorKey = "cloud_save_mirror_json";

    public static bool UseWeChatCloud { get; set; }

    public static void UploadJson(string json, Action<bool> onDone = null)
    {
        if (string.IsNullOrEmpty(json))
        {
            onDone?.Invoke(false);
            return;
        }

        // 本地镜像，便于联调与断网回退
        try
        {
            PlayerPrefs.SetString(CloudMirrorKey, json);
            PlayerPrefs.Save();
            string path = Path.Combine(Application.persistentDataPath, "cloud_mirror.json");
            File.WriteAllText(path, json);
        }
        catch (Exception e)
        {
            Debug.LogWarning("[CloudSaveBridge] 本地镜像失败: " + e.Message);
        }

        if (UseWeChatCloud)
        {
            // TODO: 微信云开发 database / storage 上传
            Debug.Log("[CloudSaveBridge] UseWeChatCloud=true, 等待微信 SDK 绑定");
            onDone?.Invoke(false);
            return;
        }

        Debug.Log("[CloudSaveBridge] 已写入本地云镜像（定版前）");
        onDone?.Invoke(true);
    }

    public static void DownloadJson(Action<string> onDone)
    {
        if (UseWeChatCloud)
        {
            // TODO: 微信云下载
            onDone?.Invoke(null);
            return;
        }
    }
}
```

----- 第 60 页 -----

```

    }

    string json = PlayerPrefs.GetString(CloudMirrorKey, null);
    if (string.IsNullOrEmpty(json))
    {
        string path = Path.Combine(Application.persistentDataPath, "cloud_mirror.json");
        if (File.Exists(path))
        {
            try { json = File.ReadAllText(path); }
            catch { /* ignore */ }
        }
    }
    onDone?.Invoke(string.IsNullOrEmpty(json) ? null : json);
}
}

// ===== FILE: Assets/Scripts/Platform/RewardedAdBridge.cs =====
using System;
using UnityEngine;

/// <summary>
/// 激励视频广告桥：正式微信 SDK 接入前用本地模拟成功。
/// </summary>
public static class RewardedAdBridge
{
    /// <summary>是否已接入真实广告 SDK（微信小游戏）。</summary>
    public static bool HasRealSdk { get; set; }

    /// <summary>
    /// 请求播放激励视频；完成后回调 success。
    /// </summary>
    public static void ShowRewarded(string placement, Action<bool> onComplete)
    {
        if (HasRealSdk)
        {
            // TODO: 微信 wx.createRewardedVideoAd(placement)
            Debug.Log($"[RewardedAdBridge] 真实 SDK 未绑定 placement={placement}，回退模拟");
        }

        #if UNITY_EDITOR || DEVELOPMENT_BUILD
            Debug.Log($"[RewardedAdBridge] 模拟激励视频成功: {placement}");
            onComplete?.Invoke(true);
        #else
            // 定版前：无 SDK 时仍允许刷新，避免卡死；上架前改为 false 或接微信
            Debug.Log($"[RewardedAdBridge] 模拟激励视频: {placement}");
            onComplete?.Invoke(true);
        #endif
    }
}

```